

算法模板从竞赛到禁赛

网络与 XCPC 算法协会

2026 年 7 月 23 日

目录

1	基础算法	4
1.1	二分答案	4
1.2	整数三分	6
2	数论	8
2.1	Miller – Rabin 素数测试与 Pollard – Rho 因式分解	8
2.2	快速幂	10
2.3	欧拉筛素数	10
2.4	扩展欧几里得 exgcd	11
2.5	模逆元	12
2.6	组合数	13
2.7	矩阵快速幂	14
2.8	线性基	16
2.9	欧拉函数	18
2.10	孙子定理 (中国剩余定理, CRT)	19
2.11	基姆拉尔森公式	20
3	计算几何	21
3.1	极角排序	21
3.2	浮点比较	22
3.3	距离, 点乘, 叉乘	22
3.4	鞋带公式	24
3.5	Pick 定理	25
3.6	平面凸包	26
4	数据结构	28
4.1	树状数组	28
4.2	线段树	29

4.3	并查集	34
4.4	分数类	35
4.5	取模类	37
4.6	自定义优先队列 (传统)	41
4.7	自定义优先队列 (lambda)	42
4.8	自定义 (多重) 集合 (传统)	43
4.9	自定义 (多重) 集合 (lambda)	44
4.10	二叉搜索树	44
5	图论	48
5.1	邻接矩阵	48
5.2	邻接表	48
5.3	DFS 深度优先搜索	52
5.4	BFS 广度优先搜索	52
5.5	01 BFS	53
5.6	Dijkstra 最短路	54
5.7	SPFA 最短路	55
5.8	Floyd 最短路	56
5.9	最小生成树 (Prim)	58
5.10	LCA (倍增)	59
5.11	LCA (离线 Tarjan)	61
5.12	SCC 强连通分量	62
6	i128 相关	65
6.1	字符串转 int 128	65
6.2	开平方	65
6.3	最大公约数	66
7	动态规划解决问题策略	67
7.1	背包 DP	67
7.1.1	01 背包	67
7.1.2	完全背包	68
7.1.3	多重背包	69
7.2	最长公共子序列 (Longest Common Subsequence)	73
7.3	状态压缩 DP	73

目录	3
8 常用函数重载	76
8.1 精确开平方	76
8.2 精准 log	76
9 字符串	78
9.1 前缀函数 (KMP)	78
9.2 字典树 (Trie)	81
9.3 AC 自动机 (Aho-Crasick)	83
10 实用工具	87
10.1 .clang-format	87
10.2 关闭流同步	87
10.3 代码计时器	88
10.4 数据生成器	88
10.5 Pair 哈希	89
10.6 Tuple 哈希	90
10.7 unordered 容器模数更改	91
10.8 字符串流	92
10.9 Lambda 自调用 (递归)	92
11 Policy-Based Data Structures	93
11.1 GCC 扩展	93
11.2 堆	93
11.3 平衡树	95

Chapter 1

基础算法

1.1 二分答案

适合于：

1. 直接求解困难；
2. 验证答案容易；
3. 答案具有单调性。

区间分为 $[\ell, mid - 1]$ 和 $[mid, r]$ (即 check 成功后 $\ell = mid$):

这种情况下 mid 需要向上取整, 否则当 $\ell = r - 1$ 且 check 返回 true 时, 会陷入死循环。

```
1 while (l < r) {
2     int mid = l + (r - l + 1) / 2; // 向上取整
3     if (check(mid)) {
4         l = mid; // [mid, r]
5     } else {
6         r = mid - 1; // [l, mid - 1]
7     }
8 }
9 return l;
```

同时下面是容器 lower_bound 的实现：

```
1 int lower_bound(vector<int>& nums, const int& target) {
2     int left = 0, right = nums.size();
3     while (left < right) {
```

```

4         int mid = left + (right - left) / 2;
5         if (nums[mid] < target)
6             left = mid + 1;
7         else
8             right = mid;
9     }
10    return left;
11 }

```

这就是容器的 `std::lower_bound` 的实现方式，返回第一个 $\geq x$ 元素的迭代器，如果值存在：返回该值的第一个位置。如果值不存在：返回比目标值 **大的第一个元素**位置。如果所有元素都小于目标值：返回 `end()` 迭代器。

那么与其相对的，`std::upper_bound` 返回第一个 $> x$ 指定值的元素的迭代器。如果值存在：跳过所有相同值，返回比目标值 **大的第一个元素**位置。如果值不存在：返回比目标值 **大的第一个元素**位置。如果所有元素都小于等于目标值：返回 `end()` 迭代器。

虽然标准库只提供了“大于”和“大于等于”的函数，但是用这两个函数，可以组合出“小于”，“小于等于”和“等于”的判定。

```

1  #include <bits/stdc++.h>
2
3  int main() {
4      std::vector<int> vec = {1, 2, 4, 4, 4, 6, 8, 10};
5      int target = 4;
6
7      // lower_bound
8      auto lower = std::lower_bound(vec.begin(), vec.end(), target);
9      std::cout << "lower_bound of " << target << " is at index: " <<
10         << (lower - vec.begin()) << std::endl;
11
12     // upper_bound
13     auto upper = std::upper_bound(vec.begin(), vec.end(), target);
14     std::cout << "upper_bound of " << target << " is at index: " <<
15         << (upper - vec.begin()) << std::endl;
16
17     // 找到最后一个小于 target 的元素
18     if (lower != vec.begin()) {
19         auto last_less = lower - 1;

```

```

18         std::cout << "Last element less than " << target << " is " <<
        ↪ *last_less << " at index " << (last_less - vec.begin()) <<
        ↪ std::endl;
19     } else {
20         std::cout << "No element is less than " << target << std::endl;
21     }
22
23     // 找到最后一个小于等于 target 的元素
24     if (upper ≠ vec.begin()) {
25         auto last_less_equal = upper - 1;
26         std::cout << "Last element less than or equal to " << target <<
        ↪ " is " << *last_less_equal << " at index " <<
        ↪ (last_less_equal - vec.begin()) << std::endl;
27     } else {
28         std::cout << "No element is less than or equal to " << target <<
        ↪ std::endl;
29     }
30
31     return 0;
32 }

```

简单总结就是：

函数	返回	若值存在	若值不存在	典型应用
<code>lower_bound</code>	第一个 $\geq$ target 的迭代器	目标值的第一个位置	第一个比它大的元素	查找元素、插入位置
<code>upper_bound</code>	第一个 $>$ target 的迭代器	跳过所有相同值的元素	第一个比它大的元素	计算元素出现次数
<code>lower_bound - 1</code>	最后一个 $<$ target 的迭代器	目标值前一个元素	最大的小于 target 的元素	反向查找小于目标值
<code>upper_bound - 1</code>	最后一个 $\leq$ target 的迭代器	目标值最后一个位置	最大的小于等于 target 的元素	反向查找小于等于目标值

1.2 整数三分

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 离散目标函数（以单峰/凹凸数组为例）
5  long long f(long long x) {
6      // 示例:  $-(x-5)^2 + 20$ , 峰值在  $x = 5$ 
7      return -(x - 5) * (x - 5) + 20;
8  }

```

```
9
10 // 整数三分模板
11 long long ternarySearchInt(long long l, long long r) {
12     // 当区间足够小 (<= 5 个点) 时直接退出, 防止整除带来的死循环或越界
13     while (r - l > 5) {
14         long long m1 = l + (r - l) / 3;
15         long long m2 = r - (r - l) / 3;
16
17         if (f(m1) < f(m2)) {
18             l = m1; // 排除 [l, m1)
19         } else {
20             r = m2; // 排除 (m2, r]
21         }
22     }
23
24     // 剩余小区间直接暴力遍历找到最高点
25     long long best_x = l;
26     long long max_val = f(l);
27     for (long long i = l + 1; i <= r; ++i) {
28         if (f(i) > max_val) {
29             max_val = f(i);
30             best_x = i;
31         }
32     }
33
34     return best_x;
35 }
36
37 int main() {
38     long long l = 0, r = 10;
39     long long best_x = ternarySearchInt(l, r);
40
41     cout << " 整数极大值点 x = " << best_x << endl;
42     cout << " 极大值 f(x) = " << f(best_x) << endl;
43
44     return 0;
45 }
```


Chapter 2

数论

2.1 Miller – Rabin 素数测试与 Pollard – Rho 因式分解

```
1 using i64 = long long;
2 i64 mul(i64 a, i64 b, i64 m) { return static_cast<__int128>(a) * b % m;
   ↪ }
3 i64 power(i64 a, i64 b, i64 m) {
4     i64 res = 1 % m;
5     for (; b; b >>= 1, a = mul(a, a, m))
6         if (b & 1) res = mul(res, a, m);
7     return res;
8 }
9 bool isprime(i64 n) {
10     if (n < 2) return false;
11     static constexpr int A[] = {2, 3, 5, 7, 11, 13, 17, 19, 23};
12     int s = __builtin_ctzll(n - 1);
13     i64 d = (n - 1) >> s;
14     for (auto a : A) {
15         if (a == n) return true;
16         i64 x = power(a, d, n);
17         if (x == 1 || x == n - 1) continue;
18         bool ok = false;
19         for (int i = 0; i < s - 1; ++i) {
20             x = mul(x, x, n);
21             if (x == n - 1) {
22                 ok = true;
23                 break;
            }
```

```

24         }
25     }
26     if (!ok) return false;
27 }
28 return true;
29 }
30 std::vector<i64> factorize(i64 n) {
31     std::vector<i64> p;
32     std::function<void(i64)> f = [&](i64 n) {
33         if (n <= 10000) {
34             for (int i = 2; i * i <= n; ++i)
35                 for (; n % i == 0; n /= i) p.push_back(i);
36             if (n > 1) p.push_back(n);
37             return;
38         }
39         if (isprime(n)) {
40             p.push_back(n);
41             return;
42         }
43         auto g = [&](i64 x) { return (mul(x, x, n) + 1) % n; };
44         i64 x0 = 2;
45         while (true) {
46             i64 x = x0;
47             i64 y = x0;
48             i64 d = 1;
49             i64 power = 1, lam = 0;
50             i64 v = 1;
51             while (d == 1) {
52                 y = g(y);
53                 ++lam;
54                 v = mul(v, std::abs(x - y), n);
55                 if (lam % 127 == 0) {
56                     d = std::gcd(v, n);
57                     v = 1;
58                 }
59                 if (power == lam) {
60                     x = y;
61                     power *= 2;

```

```

62         lam = 0;
63         d = std::gcd(v, n);
64         v = 1;
65     }
66 }
67 if (d != n) {
68     f(d);
69     f(n / d);
70     return;
71 }
72 ++x0;
73 }
74 };
75 f(n);
76 std::sort(p.begin(), p.end());
77 return p;
78 }

```

2.2 快速幂

根据二进制下的计算，将速度变成 $O(\log n)$ 并加上模数。

```

1  using i64 = long long;
2  i64 binpow(i64 a, i64 b, i64 p) {
3      i64 res = 1;
4      while (b > 0) {
5          if (b & 1) res = res * a % p;
6          a = a * a % p;
7          b >>= 1;
8      }
9      return res;
10 }

```

2.3 欧拉筛素数

暴力枚举最多要枚举 $O(\sqrt{n})$ 次，欧拉筛可以根据倍数快速确定是不是素数。

```

1  std::vector<int> minp, primes;
2

```

```

3 void sieve(int n) {
4     minp.assign(n + 1, 0);
5     primes.clear();
6
7     for (int i = 2; i <= n; i++) {
8         if (minp[i] == 0) {
9             minp[i] = i;
10            primes.push_back(i);
11        }
12
13        for (auto p : primes) {
14            if (i * p > n) {
15                break;
16            }
17            minp[i * p] = p;
18            if (p == minp[i]) {
19                break;
20            }
21        }
22    }
23 }
24
25 bool isprime(int n) { return minp[n] == n; }

```

2.4 扩展欧几里得 exgcd

解决的问题是: $ax+by = \gcd(a, b)$, $a, b \in \mathbb{N}^+$, 将其转化为 $bx+(a \bmod b)y = \gcd(b, a \bmod b)$ 。

```

1 using i64 = long long;
2 i64 exgcd(i64 a, i64 b, i64& x, i64& y) {
3     if (b == 0) {
4         x = 1;
5         y = 0;
6         return a;
7     }
8     i64 g = exgcd(b, a % b, y, x);
9     y -= a / b * x;
10    return g;

```

```

11 }
12
13 std::pair<i64, i64> sol(i64 a, i64 b, i64 m) {
14     assert(m > 0);
15     b *= -1;
16     i64 x, y;
17     i64 g = exgcd(a, m, x, y);
18     if (g < 0) {
19         g *= -1;
20         x *= -1;
21         y *= -1;
22     }
23     if (b % g != 0) {
24         return {-1, -1};
25     }
26     x = x * (b / g) % (m / g);
27     if (x < 0) {
28         x += m / g;
29     }
30     return {x, m / g};
31 }

```

2.5 模逆元

解决的问题是在模数下计算除法的时候将其转化为等价的乘法，即计算

$$\frac{a}{b} \equiv x \pmod{p} \implies ab^{p-2} \equiv x \pmod{p}$$

```

1  using i64 = long long;
2  constexpr int p = 998244353;
3  i64 binpow(i64 a, i64 b, i64 p) {
4      i64 res = 1;
5      while (b > 0) {
6          if (b & 1) res = res * a % p;
7          a = a * a % p;
8          b >>= 1;
9      }
10     return res;

```

```

11 }
12 i64 inv(i64 a, i64 p) { return binpow(a, p - 2, p); }

```

2.6 组合数

```

1  /** 组合数 (小范围预处理, 逆元 + 杨辉三角)
2   *   2024-03-14: https://qoj.ac/submission/353877
3   *   2023-10-06: https://qoj.ac/submission/203196
4   **/
5  constexpr int P = 1000000007;
6  constexpr int L = 10000;
7
8  int fac[L + 1], invfac[L + 1];
9  int sumbinom[L + 1][7];
10
11 int binom(int n, int m) {
12     if (n < m || m < 0) {
13         return 0;
14     }
15     return 1LL * fac[n] * invfac[m] % P * invfac[n - m] % P;
16 }
17
18 int power(int a, int b) {
19     int res = 1;
20     for (; b; b /= 2, a = 1LL * a * a % P) {
21         if (b % 2) {
22             res = 1LL * res * a % P;
23         }
24     }
25     return res;
26 }
27
28 int main() {
29     fac[0] = 1;
30     for (int i = 1; i <= L; i++) {
31         fac[i] = 1LL * fac[i - 1] * i % P;
32     }
33     invfac[L] = power(fac[L], P - 2);

```

```

34     for (int i = L; i; i--) {
35         invfac[i - 1] = 1LL * invfac[i] * i % P;
36     }
37
38     sumbinom[0][0] = 1;
39     for (int i = 1; i <= L; i++) {
40         for (int j = 0; j < 7; j++) {
41             sumbinom[i][j] = (sumbinom[i - 1][j] + sumbinom[i - 1][(j +
42                 ↪ 6) % 7]) % P;
43         }
44     }

```

2.7 矩阵快速幂

```

1 // https://www.luogu.com.cn/problem/P3390
2 // P3390 【模板】矩阵快速幂
3 #include <bits/stdc++.h>
4 using ll = long long;
5 using namespace std;
6
7 /**
8  * 矩阵类 - 算法竞赛优化版
9  */
10 struct Matrix {
11     int rows, cols;
12     vector<vector<long long>> mat;
13     static const int MOD = 1e9 + 7; // 根据题目需求修改模数
14
15     Matrix(int r, int c) : rows(r), cols(c), mat(r, vector<long long>(c,
16         ↪ 0)) {}
17
18     // 构造单位矩阵
19     static Matrix identity(int n) {
20         Matrix res(n, n);
21         for (int i = 0; i < n; i++) res.mat[i][i] = 1;
22         return res;

```

23 // 矩阵加法

```
24 Matrix operator+(const Matrix& b) const {
25     Matrix res(rows, cols);
26     for (int i = 0; i < rows; i++)
27         for (int j = 0; j < cols; j++) res.mat[i][j] = (mat[i][j] +
28             ↪ b.mat[i][j]) % MOD;
29     return res;
30 }
```

31 // 矩阵减法

```
32 Matrix operator-(const Matrix& b) const {
33     Matrix res(rows, cols);
34     for (int i = 0; i < rows; i++)
35         for (int j = 0; j < cols; j++) res.mat[i][j] = (mat[i][j] -
36             ↪ b.mat[i][j] + MOD) % MOD;
37     return res;
38 }
```

39 // 矩阵乘法 - 优化顺序以利用 CPU 缓存

```
40 Matrix operator*(const Matrix& b) const {
41     Matrix res(rows, b.cols);
42     for (int i = 0; i < rows; i++) {
43         for (int k = 0; k < cols; k++) { // 将 k 放在内层循环之前
44             if (mat[i][k] == 0) continue; // 稀疏矩阵小优化
45             for (int j = 0; j < b.cols; j++) {
46                 res.mat[i][j] = (res.mat[i][j] + mat[i][k] *
47                     ↪ b.mat[k][j]) % MOD;
48             }
49         }
50     }
51     return res;
52 }
```

53 // 矩阵快速幂

```
54 Matrix pow(long long exp) const {
55     Matrix res = Matrix::identity(rows);
56     Matrix base = *this;
57     while (exp > 0) {
```



```

58     while (exp > 0) {
59         if (exp % 2 == 1) res = res * base;
60         base = base * base;
61         exp /= 2;
62     }
63     return res;
64 }
65 };
66
67 int main(int argc, char const* argv[]) {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70
71     ll n, k;
72     cin >> n >> k;
73     Matrix res(n, n);
74     for (int i = 0; i < n; i++) {
75         for (int j = 0; j < n; j++) {
76             cin >> res.mat[i][j];
77         }
78     }
79     auto ress = res.pow(k);
80     for (int i = 0; i < n; i++) {
81         for (int j = 0; j < n; j++) {
82             cout << ress.mat[i][j] << " ";
83         }
84         cout << "\n";
85     }
86
87     return 0;
88 }

```

2.8 线性基

给定 n 个整数（数字可能重复），求在这些数中选取任意个，使得他们的异或和最大。

```

1 // https://www.luogu.com.cn/record/277797937
2 #include <bits/stdc++.h>

```

```
3  using namespace std;
4  using ll = long long;
5  struct LinearBasis {
6      ll d[61];
7
8      LinearBasis() {
9          for (int i = 0; i <= 60; ++i) d[i] = 0;
10     }
11
12     void insert(ll x) {
13         for (int i = 60; i >= 0; --i) {
14             if (!(x >> i)) continue;
15             if (!d[i]) {
16                 d[i] = x;
17                 return;
18             }
19             x ^= d[i];
20         }
21     }
22
23     ll query_max() {
24         ll res = 0;
25         for (int i = 60; i >= 0; --i) {
26             if ((res ^ d[i]) > res) {
27                 res ^= d[i];
28             }
29         }
30         return res;
31     }
32 };
33
34 int main() {
35     ios::sync_with_stdio(false);
36     cin.tie(nullptr);
37
38     int n;
39     cin >> n;
40     LinearBasis lb;
```

```

41     for (int i = 0; i < n; ++i) {
42         ll a;
43         cin >> a;
44         lb.insert(a);
45     }
46
47     cout << lb.query_max() << endl;
48
49     return 0;
50 }

```

2.9 欧拉函数

欧拉函数 (Euler's totient function), 即 $\varphi(n)$, 表示的是小于等于 n 和 n 互质的数的个数。

公式为:

$$\varphi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

```

1  int euler_phi(int n) {
2      int ans = n;
3      for (int i = 2; i * i <= n; i++)
4          if (n % i == 0) {
5              ans = ans / i * (i - 1);
6              while (n % i == 0) n /= i;
7          }
8      if (n > 1) ans = ans / n * (n - 1);
9      return ans;
10 }

```

在结论

$$n = \sum_{d|n} \varphi(d)$$

中代入 $n = \gcd(a, b)$, 则有

$$\gcd(a, b) = \sum_{d|\gcd(a,b)} \varphi(d) = \sum_d [d|a][d|b]\varphi(d),$$

其中 $[\cdot]$ 为 Iverson 括号。对上式求和，就可以得到

$$\sum_{i=1}^n \gcd(i, n) = \sum_d \sum_{i=1}^n [d|i][d|n] \varphi(d) = \sum_d \left\lfloor \frac{n}{d} \right\rfloor [d|n] \varphi(d) = \sum_{d|n} \left\lfloor \frac{n}{d} \right\rfloor \varphi(d).$$

这里关键的观察是 $\sum_{i=1}^n [d|i] = \left\lfloor \frac{n}{d} \right\rfloor$ ，即在 1 和 n 之间能够被 d 整除的 i 的个数是 $\left\lfloor \frac{n}{d} \right\rfloor$ 。

利用这个式子，就可以遍历约数求和了。需要多组查询的时候，可以预处理欧拉函数的前缀和，利用数论分块查询。也可以得到

$$\sum_{i=1}^n \sum_{j=1}^n \gcd(i, j) = \sum_{d=1}^n \left\lfloor \frac{n}{d} \right\rfloor^2 \varphi(d).$$

2.10 孙子定理 (中国剩余定理, CRT)

中国剩余定理 (Chinese Remainder Theorem, CRT) 可求解如下形式的一元线性同余方程组 (其中 $n_1, n_2, \dots, n_k$ 两两互质):

$$\begin{cases} x \equiv a_1 \pmod{n_1} \\ x \equiv a_2 \pmod{n_2} \\ \vdots \\ x \equiv a_k \pmod{n_k} \end{cases}$$

上面的「物不知数」问题就是一元线性同余方程组的一个实例。如下过程:

1. 计算所有模数的积 n ;

2. 对于第 i 个方程:

(a) 计算 $m_i = \frac{n}{n_i}$;

(b) 计算 m_i 在模 n_i 意义下的逆元 m_i^{-1} ;

(c) 计算 $c_i = m_i m_i^{-1}$ (**不要对 n_i 取模**)。

3. 方程组在模 n 意义下的唯一解为: $x = \sum_{i=1}^k a_i c_i \pmod{n}$ 。

```

1 using ll = long long;
2 ll CRT(int k, vector<ll>& a, vector<ll>& r) {
3     ll n = 1, ans = 0;
4     for (int i = 1; i <= k; i++) n = n * r[i];
5     for (int i = 1; i <= k; i++) {
6         ll m = n / r[i], b, y;
```

```
7         exgcd(m, r[i], b, y); //  $b * m \bmod r[i] = 1$ 
8         ans = (ans + a[i] * m * b % n) % n;
9     }
10    return (ans % n + n) % n;
11 }
```

2.11 基姆拉尔森公式

```
1  const int d[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
2
3  bool isLeap(int y) {
4      return y % 400 == 0 || (y % 4 == 0 && y % 100 != 0);
5  }
6
7  int daysInMonth(int y, int m) {
8      return d[m - 1] + (isLeap(y) && m == 2);
9  }
10
11 int getDay(int y, int m, int d) {
12     int ans = 0;
13     for (int i = 1970; i < y; i++) {
14         ans += 365 + isLeap(i);
15     }
16     for (int i = 1; i < m; i++) {
17         ans += daysInMonth(y, i);
18     }
19     ans += d;
20     return (ans + 2) % 7 + 1;
21 }
```

Chapter 3

计算几何

3.1 极角排序

```
1  #include <bits/stdc++.h>
2
3  int main() {
4      double x = 1.0;
5      double y = 1.0;
6
7      // 1. 计算弧度 (范围:  $-\pi$  到  $\pi$ )
8
9      double rad = std::atan2(y, x);
10
11     // 2. 转换为角度 (范围:  $-180^\circ$  到  $180^\circ$ )
12     // C++20 引入了 std::numbers::pi, 如果是旧版本可自定义 PI
13     constexpr double PI = 3.14159265358979323846;
14     double deg = rad * (180.0 / PI);
15
16     // 3. 如果需要  $0$  到  $360^\circ$  (或  $0$  到  $2\pi$ ) 的非负极角, 做一次简单修正:
17     double deg_360 = (deg < 0) ? (deg + 360.0) : deg;
18     double rad_2pi = (rad < 0) ? (rad + 2 * PI) : rad;
19
20     std::cout << "Rad: " << rad << " rad\n";
21     std::cout << "Deg: " << deg << "^\n";
22     // 极角
23     std::cout << "polar: " << deg_360 << "^\n";
24     return 0;
```

```
25 }
```

3.2 浮点比较

需要注意的是浮点数比较不要使用等号，而是判断精度是否在范围内：

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double eps = 1e-8;
5 int dcmp(double x, const double eps) {
6     if (fabs(x) < eps) return 0;
7     if (x < 0) return -1;
8     return 1;
9 }
```

3.3 距离，点乘，叉乘

全家桶：

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 template <class T, std::size_t N>
5 class PVector {
6 public:
7     array<T, N> data;
8     template <class... Args>
9     PVector(Args... args) : data{static_cast<T>(args)...} {
10         static_assert(sizeof...(Args) == N, "Not match!");
11     }
12     T dot(const PVector<T, N>& other) const {
13         T result = 0;
14         for (size_t i = 0; i < N; ++i) {
15             result += data[i] * other.data[i];
16         }
17         return result;
18     }
19 }
```

```

19     T distance(const PVector<T, N>& other) const {
20         T sum = 0;
21         for (size_t i = 0; i < N; ++i) {
22             T diff = data[i] - other.data[i];
23             sum += diff * diff;
24         }
25         return sqrt(sum);
26     }
27
28     PVector<T, 3> cross(const PVector<T, 3>& other) const {
29         static_assert(N == 3, "3 dim!");
30
31         return PVector<T, 3>(data[1] * other.data[2] - data[2] *
32             ↪ other.data[1],
33                               data[2] * other.data[0] - data[0] *
34             ↪ other.data[2],
35                               data[0] * other.data[1] - data[1] *
36             ↪ other.data[0]);
37     }
38
39     PVector<T, N>& operator+=(const PVector<T, N>& other) {
40         for (size_t i = 0; i < N; ++i) {
41             this->data[i] += other.data[i];
42         }
43         return *this;
44     }
45
46     PVector<T, N> operator+(const PVector<T, N>& other) const {
47         PVector<T, N> result = *this;
48         result += other;
49         return result;
50     }
51
52     PVector<T, N>& operator-=(const PVector<T, N>& other) {
53         for (size_t i = 0; i < N; ++i) {
54             this->data[i] -= other.data[i];
55         }
56         return *this;
57     }

```



```

54     }
55
56     PVector<T, N> operator-(const PVector<T, N>& other) const {
57         PVector<T, N> result = *this;
58         result -= other;
59         return result;
60     }
61     double operator*(const PVector<T, N>& other) const { return
↪     dot(other); }
62     T length() const {
63         T sum = 0;
64         for (size_t i = 0; i < N; ++i) {
65             sum += data[i] * data[i];
66         }
67         return sqrt(sum);
68     }
69
70     void print() const {
71         std::cout << "(";
72         for (std::size_t i = 0; i < N; ++i) {
73             std::cout << data[i] << (i == N - 1 ? "" : ", ");
74         }
75         std::cout << ")\n";
76     }
77 };

```

3.4 鞋带公式

先讲计算方法：在二维图形中这样一个多边形的面积是

$$\frac{1}{2} \sum_{i=1}^{n-1} \begin{vmatrix} x_i & x_{i+1} \\ y_i & y_{i+1} \end{vmatrix} = \frac{1}{2} \left(\begin{vmatrix} x_1 & x_2 \\ y_1 & y_2 \end{vmatrix} + \begin{vmatrix} x_2 & x_3 \\ y_2 & y_3 \end{vmatrix} + \cdots + \begin{vmatrix} x_{n-1} & x_n \\ y_{n-1} & y_n \end{vmatrix} \right)$$

更一般的，我们在三维空间中的模板：

```

1  template <class T>
2  T calculatePolygonArea3D(const vector<PVector<T, 3>>& vertices) {
3      if (vertices.size() < 3) return 0;
4

```

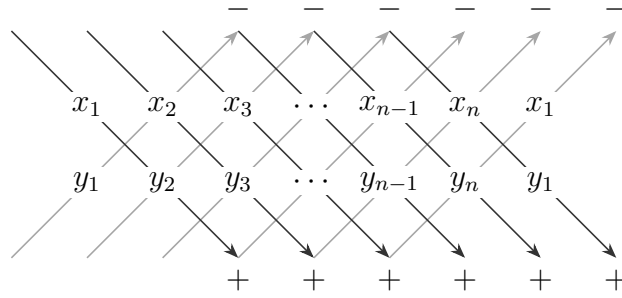


图 3.1: 鞋带公式图例

```

5   PVector<T, 3> areaVector(0, 0, 0);
6   size_t n = vertices.size();
7
8   for (size_t i = 0; i < n; ++i) {
9       const auto& p1 = vertices[i];
10      const auto& p2 = vertices[(i + 1) % n];
11
12      PVector<T, 3> crossProd = p1.cross(p2);
13      areaVector += crossProd;
14  }
15  return areaVector.length() / 2.0;
16  }

```

使用的方法为：

```

1  int main() {
2      PVector<double, 3> v1(0, 0, 0), v2(3.0, 0.0, 0.0), v3(0, 4.0, 0.0),
        ↪ v4(3.0, 4.0, 0.0);
3      vector<PVector<double, 3>> v = {v1, v2, v4, v3};
4      double dis = calculatePolygonArea3D(v);
5      cout << dis;
6  }

```

3.5 Pick 定理

Pick 定理：给定顶点均为整点的简单多边形，皮克定理说明了其面积 A 和内部格点数目 i 、边上格点数目 b 的关系： $A = i + \frac{b}{2} - 1$ 。

3.6 平面凸包

凸包是包含给定点集的最小凸集，可以形象地理解为用橡皮筋包住所有点形成的凸多边形。

```

1  /**  静态凸包 (with. Point, 新版)
2      *   2024-04-06: https://qoj.ac/submission/379920)
3      **/
4  struct Point {
5      i64 x;
6      i64 y;
7      Point() : x{0}, y{0} {}
8      Point(i64 x_, i64 y_) : x{x_}, y{y_} {}
9  };
10
11 i64 dot(Point a, Point b) {
12     return a.x * b.x + a.y * b.y;
13 }
14
15 i64 cross(Point a, Point b) {
16     return a.x * b.y - a.y * b.x;
17 }
18
19 Point operator+(Point a, Point b) {
20     return Point(a.x + b.x, a.y + b.y);
21 }
22
23 Point operator-(Point a, Point b) {
24     return Point(a.x - b.x, a.y - b.y);
25 }
26
27 auto getHull(std::vector<Point> p) {
28     std::sort(p.begin(), p.end(),
29         [&](auto a, auto b) {
30             return a.x < b.x || (a.x == b.x && a.y < b.y);
31         });
32
33     std::vector<Point> hi, lo;
34     for (auto p : p) {

```

```
35     while (hi.size() > 1 && cross(hi.back() - hi[hi.size() - 2], p -  
    ↪ hi.back()) >= 0) {  
36         hi.pop_back();  
37     }  
38     while (!hi.empty() && hi.back().x == p.x) {  
39         hi.pop_back();  
40     }  
41     hi.push_back(p);  
42     while (lo.size() > 1 && cross(lo.back() - lo[lo.size() - 2], p -  
    ↪ lo.back()) <= 0) {  
43         lo.pop_back();  
44     }  
45     if (lo.empty() || lo.back().x < p.x) {  
46         lo.push_back(p);  
47     }  
48 }  
49 return std::make_pair(hi, lo);  
50 }  
51  
52 const double inf = INFINITY;  
53
```

Chapter 4

数据结构

4.1 树状数组

维护区间值的操作，比线段树代码精简，并且支持单点修改和区间查询，使复杂度降低到 $O(\log n)$

```
1  template <typename T>
2  struct Fenwick {
3      int n;
4      std::vector<T> a;
5
6      Fenwick(int n_ = 0) { init(n_); }
7
8      void init(int n_) {
9          n = n_;
10         a.assign(n, T{});
11     }
12
13     void add(int x, const T& v) {
14         for (int i = x + 1; i <= n; i += i & -i) {
15             a[i - 1] = a[i - 1] + v;
16         }
17     }
18
19     T sum(int x) {
20         T ans{};
21         for (int i = x; i > 0; i -= i & -i) {
22             ans = ans + a[i - 1];
```

```

23     }
24     return ans;
25 }
26
27 T rangeSum(int l, int r) { return sum(r) - sum(l); }
28 // 不超过 k 的最大前缀和索引为多少，例如原数组 [1,2,3,4]，前缀和数组
29 //   ↪ [1,3,6,10]，select(5)=2
29 int select(const T& k) {
30     int x = 0;
31     T cur{};
32     for (int i = 1 << std::__lg(n); i; i /= 2) {
33         if (x + i <= n && cur + a[x + i - 1] <= k) {
34             x += i;
35             cur = cur + a[x - 1];
36         }
37     }
38     return x;
39 }
40 };
41
42 int main() {
43     Fenwick<int> f(10);
44     f.add(3, 4);
45     int res = f.rangeSum(1, 9);
46     std::cout << res; // 4
47     return 0;
48 }

```

4.2 线段树

结合了动态开点，区间修改的线段树:

```

1  using namespace std;
2  template <typename T>
3  class SegTreeLazyRangeSet {
4      vector<T> tree, lazy;
5      vector<T>* arr;
6      vector<bool> ifLazy;

```

```

7      int n, root, n4, end;
8
9      void maintain(int cl, int cr, int p) {
10         int cm = cl + (cr - cl) / 2;
11         if (cl != cr && ifLazy[p]) {
12             lazy[p * 2] = lazy[p], ifLazy[p * 2] = 1;
13             lazy[p * 2 + 1] = lazy[p], ifLazy[p * 2 + 1] = 1;
14             tree[p * 2] = lazy[p] * (cm - cl + 1);
15             tree[p * 2 + 1] = lazy[p] * (cr - cm);
16             lazy[p] = 0;
17             ifLazy[p] = 0;
18         }
19     }
20
21     T range_sum(int l, int r, int cl, int cr, int p) {
22         if (l <= cl && cr <= r) return tree[p];
23         int m = cl + (cr - cl) / 2;
24         T sum = 0;
25         maintain(cl, cr, p);
26         if (l <= m) sum += range_sum(l, r, cl, m, p * 2);
27         if (r > m) sum += range_sum(l, r, m + 1, cr, p * 2 + 1);
28         return sum;
29     }
30
31     void range_set(int l, int r, T val, int cl, int cr, int p) {
32         if (l <= cl && cr <= r) {
33             lazy[p] = val;
34             ifLazy[p] = 1;
35             tree[p] = (cr - cl + 1) * val;
36             return;
37         }
38         int m = cl + (cr - cl) / 2;
39         maintain(cl, cr, p);
40         if (l <= m) range_set(l, r, val, cl, m, p * 2);
41         if (r > m) range_set(l, r, val, m + 1, cr, p * 2 + 1);
42         tree[p] = tree[p * 2] + tree[p * 2 + 1];
43     }
44

```

```

45     void build(int s, int t, int p) {
46         if (s == t) {
47             tree[p] = (*arr)[s];
48             return;
49         }
50         int m = s + (t - s) / 2;
51         build(s, m, p * 2);
52         build(m + 1, t, p * 2 + 1);
53         tree[p] = tree[p * 2] + tree[p * 2 + 1];
54     }
55
56 public:
57     explicit SegTreeLazyRangeSet<T>(vector<T> v) {
58         n = v.size();
59         n4 = n * 4;
60         tree = vector<T>(n4, 0);
61         lazy = vector<T>(n4, 0);
62         ifLazy = vector<bool>(n4, 0);
63         arr = &v;
64         end = n - 1;
65         root = 1;
66         build(0, end, 1);
67         arr = nullptr;
68     }
69
70     void show(int p, int depth = 0) {
71         if (p > n4 || tree[p] == 0) return;
72         show(p * 2, depth + 1);
73         for (int i = 0; i < depth; ++i) putchar('\t');
74         printf("%d:%d\n", tree[p], lazy[p]);
75         show(p * 2 + 1, depth + 1);
76     }
77
78     T range_sum(int l, int r) { return range_sum(l, r, 0, end, root); }
79
80     void range_set(int l, int r, T val) { range_set(l, r, val, 0, end,
81         ↪ root); }
82 };

```


区间查询线段树:

```

1  using namespace std;
2  template <typename T>
3  class SegTreeLazyRangeMin // 类名改为更合适的 Min
4  {
5      vector<T> tree, lazy;
6      vector<T>* arr;
7      vector<bool> ifLazy;
8      int n, root, n4, end;
9
10     void maintain(int cl, int cr, int p) {
11         int cm = cl + (cr - cl) / 2;
12         if (cl != cr && ifLazy[p]) {
13             lazy[p * 2] = lazy[p], ifLazy[p * 2] = 1;
14             lazy[p * 2 + 1] = lazy[p], ifLazy[p * 2 + 1] = 1;
15             tree[p * 2] = lazy[p]; // 最小值直接等于赋值值
16             tree[p * 2 + 1] = lazy[p]; // 最小值直接等于赋值值
17             lazy[p] = 0;
18             ifLazy[p] = 0;
19         }
20     }
21
22     T range_min(int l, int r, int cl, int cr, int p) {
23         if (l <= cl && cr <= r) return tree[p];
24
25         int m = cl + (cr - cl) / 2;
26         T min_val = numeric_limits<T>::max(); // 初始化为最大值
27         maintain(cl, cr, p);
28
29         if (l <= m) min_val = min(min_val, range_min(l, r, cl, m, p *
30             ↪ 2));
31         if (r > m) min_val = min(min_val, range_min(l, r, m + 1, cr, p *
32             ↪ 2 + 1));
33
34         return min_val;
35     }
36
37     void range_set(int l, int r, T val, int cl, int cr, int p) {

```

```

36         if (l <= cl && cr <= r) {
37             lazy[p] = val;
38             ifLazy[p] = 1;
39             tree[p] = val; // 最小值直接等于赋值值
40             return;
41         }
42         int m = cl + (cr - cl) / 2;
43         maintain(cl, cr, p);
44         if (l <= m) range_set(l, r, val, cl, m, p * 2);
45         if (r > m) range_set(l, r, val, m + 1, cr, p * 2 + 1);
46
47         // 更新父节点为两个子节点的最小值
48         tree[p] = min(tree[p * 2], tree[p * 2 + 1]);
49     }
50
51     void build(int s, int t, int p) {
52         if (s == t) {
53             tree[p] = (*arr)[s];
54             return;
55         }
56         int m = s + (t - s) / 2;
57         build(s, m, p * 2);
58         build(m + 1, t, p * 2 + 1);
59
60         // 取两个子节点的最小值
61         tree[p] = min(tree[p * 2], tree[p * 2 + 1]);
62     }
63
64     public:
65         explicit SegTreeLazyRangeMin<T>(vector<T> v) {
66             n = v.size();
67             n4 = n * 4;
68             tree = vector<T>(n4, 0);
69             lazy = vector<T>(n4, 0);
70             ifLazy = vector<bool>(n4, 0);
71             arr = &v;
72             end = n - 1;
73             root = 1;

```

```

74         build(0, end, 1);
75         arr = nullptr;
76     }
77
78     void show(int p, int depth = 0) {
79         if (p > n4 || tree[p] == 0) return;
80         show(p * 2, depth + 1);
81         for (int i = 0; i < depth; ++i) putchar('\t');
82         printf("%d:%d\n", tree[p], lazy[p]);
83         show(p * 2 + 1, depth + 1);
84     }
85
86     // 查询区间最小值
87     T range_min(int l, int r) { return range_min(l, r, 0, end, root); }
88
89     // 区间赋值操作
90     void range_set(int l, int r, T val) { range_set(l, r, val, 0, end,
91         ↪ root); }
92 };

```

4.3 并查集

```

1  struct DSU {
2      std::vector<int> f, siz;
3      int setCount; // 不同集合的数量
4
5      DSU() : setCount(0) {}
6      DSU(int n) { init(n); }
7
8      void init(int n) {
9          f.resize(n);
10         std::iota(f.begin(), f.end(), 0);
11         siz.assign(n, 1);
12         setCount = n; // 初始时每个元素都是一个集合
13     }
14
15     int find(int x) {
16         while (x != f[x]) {

```

```

17         x = f[x] = f[f[x]];
18     }
19     return x;
20 }
21
22 bool same(int x, int y) { return find(x) == find(y); }
23
24 bool merge(int x, int y) {
25     x = find(x);
26     y = find(y);
27     if (x == y) {
28         return false;
29     }
30     if (siz[x] < siz[y]) {
31         std::swap(x, y);
32     }
33     siz[x] += siz[y];
34     f[y] = x;
35     setCount--; // 合并后集合数量减少 1
36     return true;
37 }
38
39 int size(int x) { return siz[find(x)]; }
40
41 // 获取不同集合的数量
42 int count() const { return setCount; }
43 };

```

4.4 分数类

如果说遇到有分数计算的可以使用分数类，这样可以直接加减乘除。

```

1  template <class T>
2  struct Frac {
3      T num;
4      T den;
5      Frac(T num_, T den_) : num(num_), den(den_) {
6          if (den < 0) {

```

```

7         den = -den;
8         num = -num;
9     }
10 }
11 Frac() : Frac(0, 1) {}
12 Frac(T num_) : Frac(num_, 1) {}
13 explicit operator double() const { return 1. * num / den; }
14 Frac& operator+=(const Frac& rhs) {
15     num = num * rhs.den + rhs.num * den;
16     den *= rhs.den;
17     return *this;
18 }
19 Frac& operator-=(const Frac& rhs) {
20     num = num * rhs.den - rhs.num * den;
21     den *= rhs.den;
22     return *this;
23 }
24 Frac& operator*=(const Frac& rhs) {
25     num *= rhs.num;
26     den *= rhs.den;
27     return *this;
28 }
29 Frac& operator/=(const Frac& rhs) {
30     num *= rhs.den;
31     den *= rhs.num;
32     if (den < 0) {
33         num = -num;
34         den = -den;
35     }
36     return *this;
37 }
38 friend Frac operator+(Frac lhs, const Frac& rhs) { return lhs +=
↪ rhs; }
39 friend Frac operator-(Frac lhs, const Frac& rhs) { return lhs -=
↪ rhs; }
40 friend Frac operator*(Frac lhs, const Frac& rhs) { return lhs *=
↪ rhs; }

```

```

41     friend Frac operator/(Frac lhs, const Frac& rhs) { return lhs /=
    ↪  rhs; }
42     friend Frac operator-(const Frac& a) { return Frac(-a.num, a.den); }
43     friend bool operator==(const Frac& lhs, const Frac& rhs) {
44         return lhs.num * rhs.den == rhs.num * lhs.den;
45     }
46     friend bool operator!=(const Frac& lhs, const Frac& rhs) {
47         return lhs.num * rhs.den != rhs.num * lhs.den;
48     }
49     friend bool operator<(const Frac& lhs, const Frac& rhs) {
50         return lhs.num * rhs.den < rhs.num * lhs.den;
51     }
52     friend bool operator>(const Frac& lhs, const Frac& rhs) {
53         return lhs.num * rhs.den > rhs.num * lhs.den;
54     }
55     friend bool operator<=(const Frac& lhs, const Frac& rhs) {
56         return lhs.num * rhs.den <= rhs.num * lhs.den;
57     }
58     friend bool operator>=(const Frac& lhs, const Frac& rhs) {
59         return lhs.num * rhs.den >= rhs.num * lhs.den;
60     }
61     friend std::ostream& operator<<(std::ostream& os, Frac x) {
62         T g = std::gcd(x.num, x.den);
63         if (x.den == g) {
64             return os << x.num / g;
65         } else {
66             return os << x.num / g << "/" << x.den / g;
67         }
68     }
69 };

```

4.5 取模类

遇到题目中可能会有对结果 998244353 取模，取模类可以限定在这个范围内，需要 cpp 20+

```

1  using i64 = long long;
2  using u64 = unsigned long long;

```

```

3  using u32 = unsigned int;
4  template <typename T>
5  constexpr T power(T a, u64 b) {
6      T res{1};
7      for (; b != 0; b /= 2, a *= a) {
8          if (b % 2 == 1) {
9              res *= a;
10         }
11     }
12     return res;
13 }
14
15 template <u32 P>
16 constexpr u32 mulMod(u32 a, u32 b) {
17     return 1ULL * a * b % P;
18 }
19
20 template <u64 P>
21 constexpr u64 mulMod(u64 a, u64 b) {
22     u64 res = a * b - u64(1.L * a * b / P - 0.5L) * P;
23     res %= P;
24     return res;
25 }
26
27 template <typename U, U P>
28     requires std::unsigned_integral<U>
29 struct ModIntBase {
30 public:
31     constexpr ModIntBase() : x{0} {}
32
33     template <typename T>
34         requires std::integral<T>
35     constexpr ModIntBase(T x_) : x{norm(x_ % T{P})} {}
36
37     constexpr static U norm(U x) {
38         if ((x >> (8 * sizeof(U) - 1) & 1) == 1) {
39             x += P;
40         }

```

```

41         if (x >= P) {
42             x -= P;
43         }
44         return x;
45     }
46
47     constexpr U val() const { return x; }
48
49     constexpr ModIntBase operator-() const {
50         ModIntBase res;
51         res.x = norm(P - x);
52         return res;
53     }
54
55     constexpr ModIntBase inv() const { return power(*this, P - 2); }
56
57     constexpr ModIntBase& operator*=(const ModIntBase& rhs) & {
58         x = mulMod<P>(x, rhs.val());
59         return *this;
60     }
61
62     constexpr ModIntBase& operator+=(const ModIntBase& rhs) & {
63         x = norm(x + rhs.x);
64         return *this;
65     }
66
67     constexpr ModIntBase& operator-=(const ModIntBase& rhs) & {
68         x = norm(x - rhs.x);
69         return *this;
70     }
71
72     constexpr ModIntBase& operator/=(const ModIntBase& rhs) & { return
    ↪ *this *= rhs.inv(); }
73
74     friend constexpr ModIntBase operator*(ModIntBase lhs, const
    ↪ ModIntBase& rhs) {
75         lhs *= rhs;
76         return lhs;

```



```
77     }
78
79     friend constexpr ModIntBase operator+(ModIntBase lhs, const
    ↪ ModIntBase& rhs) {
80         lhs += rhs;
81         return lhs;
82     }
83
84     friend constexpr ModIntBase operator-(ModIntBase lhs, const
    ↪ ModIntBase& rhs) {
85         lhs -= rhs;
86         return lhs;
87     }
88
89     friend constexpr ModIntBase operator/(ModIntBase lhs, const
    ↪ ModIntBase& rhs) {
90         lhs /= rhs;
91         return lhs;
92     }
93
94     friend constexpr std::ostream& operator<<(std::ostream& os, const
    ↪ ModIntBase& a) {
95         return os << a.val();
96     }
97
98     friend constexpr bool operator==(ModIntBase lhs, ModIntBase rhs) {
99         return lhs.val() == rhs.val();
100     }
101
102     friend constexpr bool operator!=(ModIntBase lhs, ModIntBase rhs) {
103         return lhs.val() != rhs.val();
104     }
105
106     friend constexpr bool operator<(ModIntBase lhs, ModIntBase rhs) {
107         return lhs.val() < rhs.val();
108     }
109
110     private:
```

```

111     U x;
112 };
113
114 template <u32 P>
115 using ModInt = ModIntBase<u32, P>;
116
117 template <u64 P>
118 using ModInt64 = ModIntBase<u64, P>;
119
120 constexpr u32 P = 998244353;
121 using Z = ModInt<P>;

```

4.6 自定义优先队列（传统）

```

1  using namespace std;
2  priority_queue<int> q; // 默认从大到小（大根堆）
3  priority_queue<int, vector<int>, less<int>> q; // 从大到小排序（大根堆）
4  priority_queue<int, vector<int>, greater<int>> q; // 从小到大排序（小根堆）
5  struct Student {
6      string name;
7      int score;
8      int age;
9  };
10
11 // 按分数降序排列
12 struct CompareByScore {
13     bool operator()(const Student& a, const Student& b) {
14         return a.score < b.score; // 最大堆：返回 true 表示 a 的优先级低于 b
15     }
16 };
17
18 // 按分数升序排列
19 struct CompareByScoreAsc {
20     bool operator()(const Student& a, const Student& b) {
21         return a.score > b.score; // 最小堆
22     }
23 };
24

```

```

25 // 多级排序：先按分数降序，分数相同按年龄升序
26 struct CompareMulti {
27     bool operator()(const Student& a, const Student& b) {
28         if (a.score != b.score) return a.score < b.score; // 分数高的在前
29         return a.age > b.age; // 年龄小的在前
30     }
31 };
32
33 int main() {
34     // 使用自定义比较类
35     priority_queue<Student, vector<Student>, CompareByScore> pq1;
36
37     // 或者使用多级排序
38     priority_queue<Student, vector<Student>, CompareMulti> pq2;
39
40     pq2.push({"Alice", 85, 20});
41     pq2.push({"Bob", 85, 19});
42     pq2.push({"Charlie", 90, 21});
43
44     while (!pq2.empty()) {
45         auto s = pq2.top();
46         cout << s.name << " " << s.score << " " << s.age << endl;
47         pq2.pop();
48     }
49
50     return 0;
51 }

```

4.7 自定义优先队列 (lambda)

```

1 using namespace std;
2
3 struct Student {
4     string name;
5     int score;
6 };
7
8 int main() {

```

```

9      // 使用 lambda 表达式作为比较函数
10     auto cmp = [](const Student& a, const Student& b) {
11         return a.score < b.score; // 最大堆
12     };
13
14     // 注意：需要指定容器类型和比较器类型
15     priority_queue<Student, vector<Student>, decltype(cmp)> pq(cmp);
16
17     // 也可以这样定义（更简洁的方式）
18     priority_queue<Student, vector<Student>, function<bool(const
19         ↪ Student&, const Student&>> pq2(
20         [](const Student& a, const Student& b) {
21             return a.score > b.score; // 最小堆
22         });
23
24     pq.push({"Alice", 85});
25     pq.push({"Bob", 92});
26
27     return 0;

```

4.8 自定义（多重）集合（传统）

```

1  using namespace std;
2  // 注意和优先队列是相反的
3  set<int> s; //默认从小到大
4  set<int, vector<int>, less<int>> q; //从小到大排序
5  set<int, vector<int>, greater<int>> q; //从大到小排序
6  struct Person {
7      string name;
8      int age;
9  };
10
11  // 自定义比较器
12  struct PersonComparator {
13      bool operator()(const Person& a, const Person& b) const {
14          // 先按年龄降序，年龄相同按名字升序
15          if (a.age != b.age)

```

```

16         return a.age > b.age; // 注意这里是 >, 表示降序
17         return a.name < b.name;
18     }
19 };
20
21 // 使用
22 set<Person, PersonComparator> people1;
23 multiset<Person, PersonComparator> people2;

```

4.9 自定义（多重）集合（lambda）

```

1  using namespace std;
2  struct Person {
3      string name;
4      int age;
5  };
6
7  // 使用 decltype 和 lambda 定义比较器
8  auto cmp = [](const Person& a, const Person& b) {
9      return a.age < b.age; // 只按年龄升序
10 };
11
12 // 注意: lambda 需要指定模板参数
13 set<Person, decltype(cmp)> people(cmp);
14 multiset<Person, decltype(cmp)> people_multi(cmp);

```

4.10 二叉搜索树

这个搜索树没有什么用处，但是是平衡树的绝对基础。

二叉搜索树满足中序遍历从小到大排列。

```

1  #include <iostream>
2  #include <functional>
3
4  template <typename T>
5  class BST {
6  private:
7      struct Node {

```

```
8      T data;
9      Node* left;
10     Node* right;
11
12     Node(const T& value) : data(value), left(nullptr),
13     ↪ right(nullptr) {}
14
15     };
16
17     Node* root;
18
19     // 辅助：销毁树
20     void clear(Node* node) {
21         if (!node) return;
22         clear(node->left);
23         clear(node->right);
24         delete node;
25     }
26
27     // 辅助：插入
28     Node* insert(Node* node, const T& value) {
29         if (!node) return new Node(value);
30         if (value < node->data)
31             node->left = insert(node->left, value);
32         else if (value > node->data)
33             node->right = insert(node->right, value);
34         return node;
35     }
36
37     // 辅助：查找最小值节点（用于删除）
38     Node* findMin(Node* node) {
39         while (node && node->left) node = node->left;
40         return node;
41     }
42
43     // 辅助：删除节点
44     Node* remove(Node* node, const T& value) {
```

```

45     if (value < node->data) {
46         node->left = remove(node->left, value);
47     } else if (value > node->data) {
48         node->right = remove(node->right, value);
49     } else {
50         // 找到节点了
51         if (!node->left) { // 只有右子树或无子树
52             Node* temp = node->right;
53             delete node;
54             return temp;
55         } else if (!node->right) { // 只有左子树
56             Node* temp = node->left;
57             delete node;
58             return temp;
59         }
60         // 有两个子节点：用右子树的最小值替代当前节点
61         Node* temp = findMin(node->right);
62         node->data = temp->data;
63         node->right = remove(node->right, temp->data);
64     }
65     return node;
66 }
67
68 // 辅助：中序遍历 (BST 的中序遍历是有序的)
69 void inorder(Node* node) {
70     if (!node) return;
71     inorder(node->left);
72     std::cout << node->data << " ";
73     inorder(node->right);
74 }
75
76 public:
77     BST() : root(nullptr) {}
78     ~BST() { clear(root); }
79
80     void insert(const T& value) { root = insert(root, value); }
81     void remove(const T& value) { root = remove(root, value); }
82

```

```
83     bool search(const T& value) {
84         Node* curr = root;
85         while (curr) {
86             if (value == curr->data) return true;
87             curr = (value < curr->data) ? curr->left : curr->right;
88         }
89         return false;
90     }
91
92     void display() {
93         inorder(root);
94         std::cout << std::endl;
95     }
96 };
```


Chapter 5

图论

5.1 邻接矩阵

使用一个二维数组 `adj` 来存边，其中 `adj[u][v]` 为 1 表示存在 u 到 v 的边，为 0 表示不存在。如果是带边权的图，可以在 `adj[u][v]` 中存储 u 到 v 的边的边权。

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      // 初始化  $n$  行  $m$  列，所有元素为 0
6      int n = 3, m = 4;
7      vector<vector<int>> matrix(n, vector<int>(m, 0));
8
9      for (int i = 1; i <= m; ++i) {
10         int u, v, p;
11         cin >> u >> v >> p;
12         matrix[u][v] = p; // 从  $u$  到  $v$  权值为  $p$ 
13     }
14 }
```

5.2 邻接表

由于邻接表在附带信息存图中很有用处，所以后面的算法模板都以邻接表为例（虽然副社长 Timothyst 最喜欢的是链式前向星存图）。

假设给定下面的这个图：

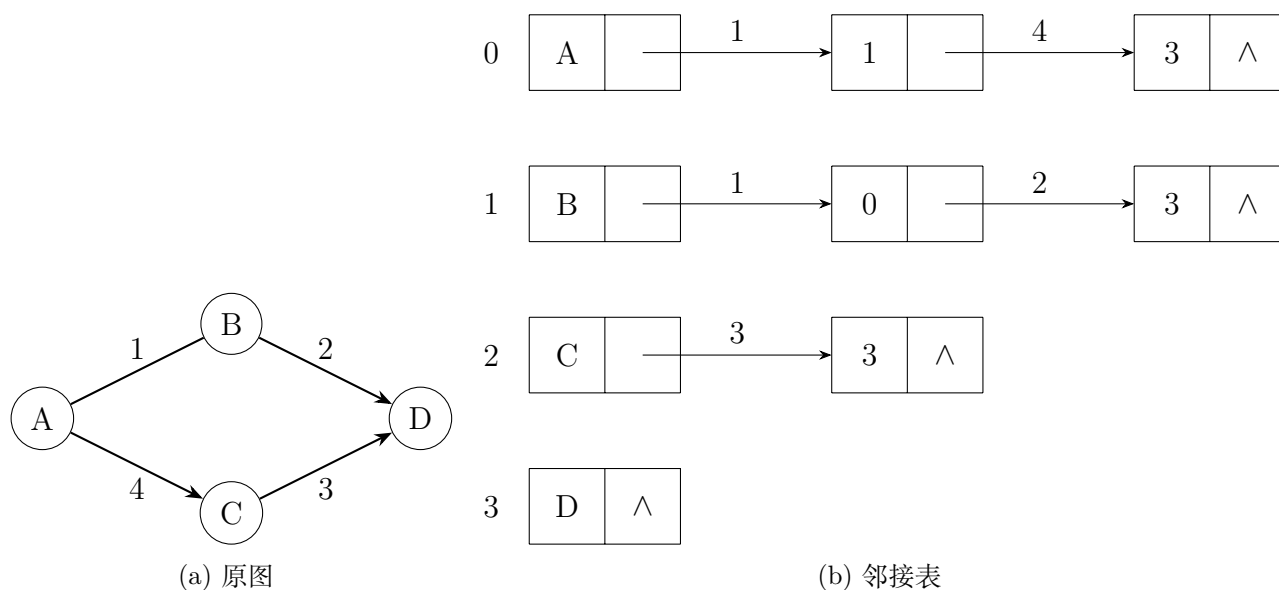


图 5.1: 邻接表图示

下面是具体代码实现过程，使用 `class` 封装，简单的添加信息在 `createGraph()` 函数内。如果使用 `class` 建图，参照 `main` 函数。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  // 竞赛用图的简化实现
5  class Graph {
6  private:
7      int n; // 顶点数
8      vector<vector<pair<int, int>>> adj; // 邻接表 (to, weight)
9      vector<bool> directed; // 标记边是否有向
10
11 public:
12     // 构造函数：初始化 n 个顶点
13     Graph(int size) : n(size), adj(size), directed(size) {}
14
15     // 添加有向边
16     void addDirectedEdge(int u, int v, int w = 1) {
17         adj[u].push_back({v, w});
18         directed[u] = true;
19     }
20

```

```

21 // 添加无向边
22 void addUndirectedEdge(int u, int v, int w = 1) {
23     adj[u].push_back({v, w});
24     adj[v].push_back({u, w});
25     directed[u] = false;
26     directed[v] = false;
27 }
28
29 // 获取邻接表
30 const vector<vector<pair<int, int>>>& getAdj() const { return adj; }
31
32 // 获取顶点数
33 int getSize() const { return n; }
34
35 // 打印图 (调试用)
36 void print() {
37     for (int i = 0; i < n; i++) {
38         if (!adj[i].empty()) {
39             cout << i << " -> ";
40             for (auto [v, w] : adj[i]) {
41                 cout << "(" << v << ", " << w << ") ";
42             }
43             cout << endl;
44         }
45     }
46 }
47
48 vector<tuple<int, int, int>> sortAdjByWeight() {
49     vector<tuple<int, int, int>> vtiii;
50     for (int i = 0; i < n; i++) {
51         for (auto j : adj[i]) {
52             tuple<int, int, int> tmp(i, j.first, j.second);
53             vtiii.push_back(tmp);
54         }
55     }
56     sort(vtiii.begin(), vtiii.end(),
57         [](const tuple<int, int, int>& a, const tuple<int, int,
58             ↪ int>& b) {

```

```

58         return get<2>(a) < get<2>(b);
59     });
60     return vtiii;
61 }
62 bool operator==(const Graph& b) {
63     if (b.getAdj() == adj) return true;
64     return false;
65 }
66 };
67
68 vector<vector<pair<int, int>>> createGraph() {
69     // 创建 4 个顶点的图 (对应 A=0, B=1, C=2, D=3)
70     int n = 4;
71     vector<vector<pair<int, int>>> adj(n);
72
73     // 添加边 (按照题目图示)
74     // A(0)-B(1) 无向边, 权重 1
75     adj[0].push_back({1, 1});
76     adj[1].push_back({0, 1});
77
78     // A(0)->C(2) 有向边, 权重 4
79     adj[0].push_back({2, 4});
80
81     // B(1)->D(3) 有向边, 权重 2
82     adj[1].push_back({3, 2});
83
84     // C(2)->D(3) 有向边, 权重 3
85     adj[2].push_back({3, 3});
86
87     return adj;
88 }
89 int main() {
90     Graph g(4); // 4 个顶点
91     g.addUndirectedEdge(0, 1, 1); // A-B 无向
92     g.addDirectedEdge(0, 2, 4); // A->C 有向
93     g.addDirectedEdge(1, 3, 2); // B->D 有向
94     g.addDirectedEdge(2, 3, 3); // C->D 有向
95

```

```

96     g.print();
97 }

```

5.3 DFS 深度优先搜索

```

1  void dfs(int u, const vector<vector<pair<int, int>>> &adj, vector<bool>
    ↪ &visited)
2  {
3      visited[u] = true;
4      cout << u << " ";
5
6      for (auto [v, w] : adj[u])
7      {
8          if (!visited[v])
9          {
10             dfs(v, adj, visited);
11         }
12     }
13 }

```

5.4 BFS 广度优先搜索

```

1  void bfs(int start, const vector<vector<pair<int, int>>>& adj) {
2      int n = adj.size();
3      vector<bool> visited(n, false);
4      queue<int> q;
5
6      visited[start] = true;
7      q.push(start);
8
9      while (!q.empty()) {
10         int u = q.front();
11         q.pop();
12         cout << u << " ";
13
14         for (auto [v, w] : adj[u]) {
15             if (!visited[v]) {
16                 visited[v] = true;

```

```

17         q.push(v);
18     }
19 }
20 }
21 cout << endl;
22 }
```

5.5 01 BFS

```

1  vector<int> zeroOneBFS(const Graph& g, int start) {
2      int n = g.getSize();
3      const auto& adj = g.getAdj();
4
5      // 1. 初始化距离数组为 INT_MAX, 起点距离为 0
6      vector<int> dist(n, INT_MAX);
7      dist[start] = 0;
8
9      // 2. 双端队列维护 0-1 层的节点单调性
10     deque<int> dq;
11     dq.push_back(start);
12
13     while (!dq.empty()) {
14         int u = dq.front();
15         dq.pop_front();
16
17         // 遍历 u 的所有邻居 (v, w)
18         for (const auto& [v, w] : adj[u]) {
19             // 松弛操作 (Relaxation)
20             if (dist[u] + w < dist[v]) {
21                 dist[v] = dist[u] + w;
22
23                 // 核心逻辑: 边权 0 插队头, 边权 1 插队尾
24                 if (w == 0) {
25                     dq.push_front(v);
26                 } else {
27                     dq.push_back(v);
28                 }
29             }
30         }
31     }
32 }

```

```
30         }
31     }
32 }
33 return dist;
34 }
```

5.6 Dijkstra 最短路

采用优先队列最短路，但是边权不能为负。

```
1 vector<int> dijkstra(int start, const vector<vector<pair<int, int>>>&
  ↪ adj) {
2     int n = adj.size();
3     vector<int> dist(n, INT_MAX);
4     priority_queue<pair<int, int>, vector<pair<int, int>>,
  ↪ greater<pair<int, int>>> pq;
5
6     dist[start] = 0;
7     pq.push({0, start});
8
9     while (!pq.empty()) {
10         auto [d, u] = pq.top();
11         pq.pop();
12
13         if (d > dist[u]) continue;
14
15         for (auto [v, w] : adj[u]) {
16             if (dist[v] > dist[u] + w) {
17                 dist[v] = dist[u] + w;
18                 pq.push({dist[v], v});
19             }
20         }
21     }
22
23     return dist;
24 }
```

5.7 SPFA 最短路

SPFA 可以用来算带有负权值的最短路,但是不能用于负环最短路,如下图所示, $E \rightarrow A$ 权值为 -20 , 如果 $A \rightarrow C \rightarrow D \rightarrow E \rightarrow A$ 这样循环走, 负环上的最短路应当为无穷小。

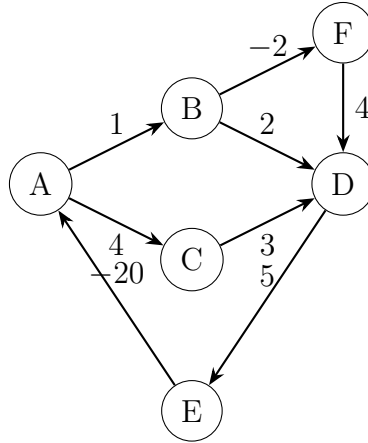


图 5.2: 负环

```

1  vector<int> spfa(int start, const vector<vector<pair<int, int>>>& adj) {
2      int n = adj.size();
3      vector<int> dist(n, INT_MAX);
4      vector<bool> inQueue(n, false);
5      vector<int> cnt(n, 0); // 记录入队次数, 用于检测负环
6
7      dist[start] = 0;
8      queue<int> q;
9      q.push(start);
10     inQueue[start] = true;
11     cnt[start]++;
12
13     while (!q.empty()) {
14         int u = q.front();
15         q.pop();
16         inQueue[u] = false;
17
18         for (auto [v, w] : adj[u]) {
19             if (dist[u] != INT_MAX && dist[v] > dist[u] + w) {
20                 dist[v] = dist[u] + w;
21

```



```

22         // 如果 v 不在队列中，则加入队列
23         if (!inQueue[v]) {
24             q.push(v);
25             inQueue[v] = true;
26             cnt[v]++;
27
28             // 检测负环：如果一个节点入队超过 n 次，说明存在负环
29             if (cnt[v] > n) {
30                 // 存在负环，返回空 vector 或特殊标记
31                 return vector<int>();
32             }
33         }
34     }
35 }
36 }
37
38 return dist;
39 }

```

5.8 Floyd 最短路

多源最短路，Floyd 算法本质上是一个动态规划算法。定义状态 $dp[k][i][j]$ ：只允许使用前 k 个节点（节点 $1 \dots k$ ）作为中转站时，从 i 到 j 的最短距离。转移方程如下：

$$dp[k][i][j] = \min(dp[k-1][i][j], dp[k-1][i][k] + dp[k-1][k][j])$$

1. 不通过节点 k 中转：距离维持上一轮的 $dp[k-1][i][j]$ 。
2. 通过节点 k 中转：拆分为 $i \rightarrow k$ 和 $k \rightarrow j$ 两段路径之和。当进行空间优化（省去第一维 k ）后，就变成了我们熟悉的二维矩阵更新。

```

1  vector<vector<int>> floyd(const Graph& g) {
2      int n = g.getSize();
3      const auto& adj = g.getAdj();
4
5      // 1. 初始化距离矩阵：点到自身为 0，其余初始化为 INF
6      vector<vector<int>> dist(n, vector<int>(n, INF));
7      for (int i = 0; i < n; i++) {
8          dist[i][i] = 0;

```

```

9      }
10
11     // 2. 读入图中既有的边 (处理多重边时取最小值)
12     for (int u = 0; u < n; u++) {
13         for (auto [v, w] : adj[u]) {
14             dist[u][v] = min(dist[u][v], w);
15         }
16     }
17
18     // 3. 核心动态规划: k 在最外层
19     for (int k = 0; k < n; k++) {
20         for (int i = 0; i < n; i++) {
21             for (int j = 0; j < n; j++) {
22                 // 防止 INF 加算导致的整型溢出
23                 if (dist[i][k] != INF && dist[k][j] != INF) {
24                     dist[i][j] = min(dist[i][j], dist[i][k] +
25                                     ↪ dist[k][j]);
26                 }
27             }
28         }
29
30     return dist;
31 }
32
33 // 辅助打印最短路径矩阵的函数
34 void printShortestPaths(const vector<vector<int>>& dist) {
35     int n = dist.size();
36     cout << "\n最短路径矩阵 (Floyd 结果):\n\t";
37     for (int i = 0; i < n; i++) cout << "[" << i << "]\t";
38     cout << endl;
39
40     for (int i = 0; i < n; i++) {
41         cout << "[" << i << "]\t";
42         for (int j = 0; j < n; j++) {
43             if (dist[i][j] == INF) cout << "INF\t";
44             else cout << dist[i][j] << "\t";
45         }

```

```

46         cout << endl;
47     }
48 }

```

5.9 最小生成树 (Prim)

请先学会并查集与图形建立后再解决这个问题。

最小生成树解决的问题就是选择边使其权和最小的生成树，也就是说要满足 2 点：

1. 生成的图是一颗树（不存在环）；
2. 边权总和最小。

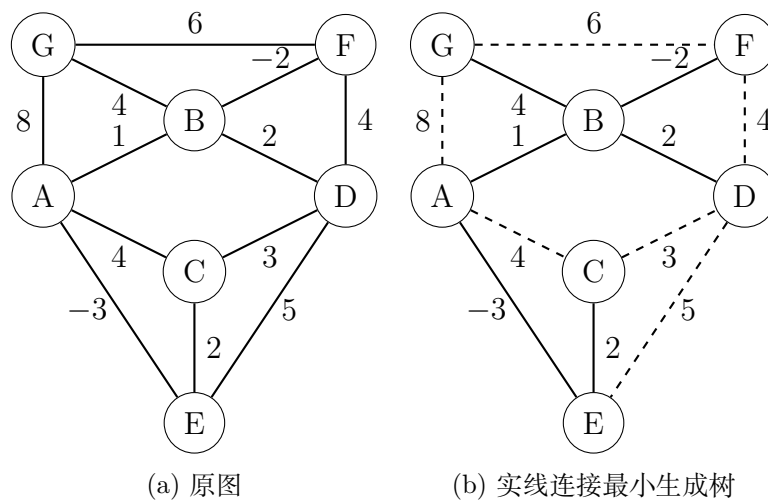


图 5.3: 最小生成树

需要注意的是，这个最小生成树需要保证是一个联通块，如果不保证联通，请使用并查集或 DFS 等方法搜索一下是否完全包含整个图。

```

1 Graph prim(Graph g, int& weight) {
2     using tiii = tuple<int, int, int>;
3     DSU dsu(g.getSize());
4     auto cmp = [](const tiii& a, const tiii& b) { return get<2>(a) >
5         ↪ get<2>(b); };
6     priority_queue<tiii, vector<tiii>, decltype(cmp)> pq(cmp);
7     Graph newg(g.getSize());
8     auto adj = g.getAdj();
9     // 可以从 1 开始

```

```

9      int cur = 0;
10     for (auto j : adj[cur]) {
11         tiii tmp(cur, j.first, j.second);
12         pq.push(tmp);
13     }
14     while (!pq.empty()) {
15         tiii top = pq.top();
16         pq.pop();
17         if (!dsu.same(get<0>(top), get<1>(top))) {
18             dsu.merge(get<0>(top), get<1>(top));
19             newg.addUndirectedEdge(get<0>(top), get<1>(top),
20                                     ↪ get<2>(top));
21             weight += get<2>(top);
22             cur = get<1>(top);
23             for (auto j : adj[cur]) {
24                 tiii tmp(cur, j.first, j.second);
25                 pq.push(tmp);
26             }
27         }
28     }
29     return newg;

```

5.10 LCA (倍增)

倍增法的核心是利用 2^k 的跳跃来快速逼近祖先。我们需要维护一个 $fa[u][k]$ 数组，表示从节点 u 向上跳 2^k 步到达的祖先。

$$fa[u][k] = fa[fa[u][k-1]][k-1]$$

```

1  class LCA_BinaryLifting {
2  private:
3      int n, logn;
4      vector<int> depth;
5      vector<vector<int>> fa;
6
7  public:
8      LCA_BinaryLifting(const Graph& g, int root = 1) {
9          n = g.getSize();

```

```

10     logn = log2(n) + 1;
11     depth.assign(n, 0);
12     fa.assign(n, vector<int>(logn, 0));
13
14     auto& adj = g.getAdj();
15     bfs(root, adj);
16 }
17
18 void bfs(int root, const vector<vector<pair<int, int>>>& adj) {
19     queue<int> q;
20     q.push(root);
21     depth[root] = 1; // 深度通常从 1 开始, 0 预留给哨兵
22
23     while (!q.empty()) {
24         int u = q.front();
25         q.pop();
26
27         for (auto [v, w] : adj[u]) {
28             if (!depth[v]) {
29                 depth[v] = depth[u] + 1;
30                 fa[v][0] = u;
31                 for (int k = 1; k < logn; ++k) {
32                     fa[v][k] = fa[fa[v][k - 1]][k - 1];
33                 }
34                 q.push(v);
35             }
36         }
37     }
38 }
39
40 int query(int u, int v) {
41     if (depth[u] < depth[v]) swap(u, v);
42
43     // 1. 先跳到同一高度
44     for (int k = logn - 1; k >= 0; --k) {
45         if (depth[fa[u][k]] >= depth[v]) {
46             u = fa[u][k];
47         }

```

```

48     }
49     if (u == v) return u;
50
51     // 2. 一起向上跳
52     for (int k = logn - 1; k >= 0; --k) {
53         if (fa[u][k] != fa[v][k]) {
54             u = fa[u][k];
55             v = fa[v][k];
56         }
57     }
58     return fa[u][0];
59 }
60 };

```

5.11 LCA (离线 Tarjan)

如果图是静态的, Tarjan 离线算法是 $O(V + E + Q)$ 的最优选。它利用并查集 (DSU) 在 DFS 过程中实时合并已访问的分支。

```

1  class LCA_Tarjan {
2  private:
3      vector<int> parent;
4      vector<bool> visited;
5      vector<int> results;
6      struct Query {
7          int to, id;
8      };
9      vector<vector<Query>> queries;
10
11     int find(int i) { return parent[i] == i ? i : parent[i] =
        ↪ find(parent[i]); }
12
13 public:
14     LCA_Tarjan(int n, int q_count) : parent(n), visited(n, false),
        ↪ results(q_count), queries(n) {
15         iota(parent.begin(), parent.end(), 0);
16     }
17

```

```

18     void addQuery(int u, int v, int id) {
19         queries[u].push_back({v, id});
20         queries[v].push_back({u, id});
21     }
22
23     void solve(int u, const vector<vector<pair<int, int>>>& adj) {
24         visited[u] = true;
25         for (auto [v, w] : adj[u]) {
26             if (!visited[v]) {
27                 solve(v, adj);
28                 parent[v] = u; // 合并到父节点
29             }
30         }
31         for (auto& q : queries[u]) {
32             if (visited[q.to]) { // 如果另一个点也访问过了
33                 results[q.id] = find(q.to);
34             }
35         }
36     }
37
38     int getResult(int id) { return results[id]; }
39 };

```

5.12 SCC 强连通分量

```

1  class TarjanSolver {
2  private:
3      int n;
4      int timestamp;
5      int scc_cnt;
6      vector<int> dfn, low, scc_id;
7      vector<bool> in_stack;
8      stack<int> stk;
9
10     void dfs(int u, const vector<vector<pair<int, int>>>& adj) {
11         dfn[u] = low[u] = ++timestamp;
12         stk.push(u);
13         in_stack[u] = true;

```

```

14
15     for (auto& edge : adj[u]) {
16         int v = edge.first;
17         if (!dfn[v]) {
18             dfs(v, adj);
19             low[u] = min(low[u], low[v]);
20         } else if (in_stack[v]) {
21             low[u] = min(low[u], dfn[v]);
22         }
23     }
24
25     if (dfn[u] == low[u]) {
26         scc_cnt++;
27         while (true) {
28             int v = stk.top();
29             stk.pop();
30             in_stack[v] = false;
31             scc_id[v] = scc_cnt; // SCC 编号从 1 开始
32             if (u == v) break;
33         }
34     }
35 }
36
37 public:
38     TarjanSolver(int size) : n(size), timestamp(0), scc_cnt(0) {
39         dfn.assign(n, 0);
40         low.assign(n, 0);
41         scc_id.assign(n, 0);
42         in_stack.assign(n, false);
43     }
44
45     // 执行缩点并返回新图
46     Graph condense(const Graph& g) {
47         const auto& adj = g.getAdj();
48
49         // 1. 求出所有 SCC
50         for (int i = 0; i < n; i++) {
51             if (!dfn[i]) dfs(i, adj);

```



```
52     }
53
54     // 2. 建立新图：新图的顶点数是 scc_cnt
55     // 注意：由于你的 Graph 类下标可能从 0 开始，这里新图大小设为 scc_cnt + 1
56     Graph condensedGraph(scc_cnt + 1);
57
58     // 3. 遍历原图所有的边进行映射
59     for (int u = 0; u < n; u++) {
60         for (auto& edge : adj[u]) {
61             int v = edge.first;
62             int w = edge.second;
63
64             // 如果端点不在同一个 SCC 中，连接这两个 SCC
65             if (scc_id[u] != scc_id[v]) {
66                 condensedGraph.addDirectedEdge(scc_id[u], scc_id[v],
67                     ↪ w);
68             }
69         }
70
71     }
72     return condensedGraph;
73 }
74
75 // 获取某个点所属的 SCC 编号
76 int getID(int u) { return scc_id[u]; }
```

Chapter 6

i128 相关

6.1 字符串转 int 128

```
1  using i128 = __int128;
2  i128 toi128(const std::string& s) {
3      i128 n = 0;
4      for (auto c : s) {
5          n = n * 10 + (c - '0');
6      }
7      return n;
8  }
```

6.2 开平方

```
1  using i128 = __int128;
2  i128 sqrti128(i128 n) {
3      i128 lo = 0, hi = 1E16;
4      while (lo < hi) {
5          i128 x = (lo + hi + 1) / 2;
6          if (x * x <= n) {
7              lo = x;
8          } else {
9              hi = x - 1;
10         }
11     }
12     return lo;
13 }
```

6.3 最大公约数

```
1  using i128 = __int128;  
2  i128 gcd(i128 a, i128 b) {  
3      while (b) {  
4          a %= b;  
5          std::swap(a, b);  
6      }  
7      return a;  
8  }
```

Chapter 7

动态规划解决问题策略

7.1 背包 DP

此类问题和现实题型讲解。

7.1.1 01 背包

有 N 件物品和一个容量为 V 的背包。第 i 件物品的费用是 $v[i]$ ，价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量，且价值总和最大。

这是最基础的背包问题，特点是：每种物品仅有一件，可以选择放或不放，接下来就是设计状态，设 $f[i][v]$ 表示选择到第 i 个物品时，背包放入了累计 v 的最大价值。

这样就有状态转移方程：

$$f[i][v] = \max f[i-1][v], f[i-1][v-v[i]] + w[i]$$

每多拿一个物品，就从 0 到 n （背包的大小）计算当前大小的背包可以放置的最大价值。将拿这个物品与不拿这个物品的最大价值做比较。若拿这个物品可以增大最大价值，就从没有拿这个物品的且正好有一个可以放这个物体体积的那个值转移过来。若不拿，则直接顺承原来的值。

```
1  /* 0-1 背包：动态规划 */
2  int knapsackDP(vector<int> &wgt, vector<int> &val, int cap) {
3      int n = wgt.size();
4      // 初始化 dp 表
5      vector<vector<int>> dp(n + 1, vector<int>(cap + 1, 0));
6      // 状态转移
7      for (int i = 1; i <= n; i++) {
8          for (int c = 1; c <= cap; c++) {
```

```

9         if (wgt[i - 1] > c) {
10             // 若超过背包容量, 则不选物品 i
11             dp[i][c] = dp[i - 1][c];
12         } else {
13             // 不选和选物品 i 这两种方案的较大值
14             dp[i][c] = max(dp[i - 1][c], dp[i - 1][c - wgt[i - 1]] +
15                 ↪ val[i - 1]);
16         }
17     }
18     return dp[n][cap];
19 }
20 /* 0-1 背包: 空间优化后的动态规划 */
21 int knapsackDPComp(vector<int> &wgt, vector<int> &val, int cap) {
22     int n = wgt.size();
23     // 初始化 dp 表
24     vector<int> dp(cap + 1, 0);
25     // 状态转移
26     for (int i = 1; i <= n; i++) {
27         // 倒序遍历
28         for (int c = cap; c >= 1; c--) {
29             if (wgt[i - 1] <= c) {
30                 // 不选和选物品 i 这两种方案的较大值
31                 dp[c] = max(dp[c], dp[c - wgt[i - 1]] + val[i - 1]);
32             }
33         }
34     }
35     return dp[cap];
36 }

```

7.1.2 完全背包

有 N 种物品和一个容量为 V 的背包, 每种物品都有无限件可用。第 i 种物品的费用是 $v[i]$, 价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量, 且价值总和最大。

这个问题非常类似于 0-1 背包问题, 所不同的是每种物品有无限件。也就是从每种物品的角度考虑, 与它相关的策略已并非取或不取两种, 而是有取 0 件、取 1 件、取 2 件……等很多种。如果仍然按照 01 背包的思路, 转移方程为:

$$f[i][v] = \max f[i-1][v], f[i][v - v[i-1]] + w[i-1]$$

```

1  int unboundedKnapsackDP(vector<int> &wgt, vector<int> &val, int cap)
2  {
3      int n = wgt.size();
4      vector<vector<int>> dp(n + 1, vector<int>(cap + 1, 0));
5      for (int i = 1; i <= n; i++)
6      {
7          for (int c = 1; c <= cap; c++)
8          {
9              if (wgt[i - 1] > c)
10             {
11                 // 若超过背包容量, 则不选物品 i
12                 dp[i][c] = dp[i - 1][c];
13             }
14             else
15             {
16                 // 不选和选物品 i 这两种方案的较大值
17                 dp[i][c] = max(dp[i - 1][c], dp[i][c - wgt[i - 1]] +
18                               ↪ val[i - 1]);
19             }
20         }
21     }
22     return dp[n][cap];
23 }
```

7.1.3 多重背包

有 N 种物品和一个容量为 V 的背包。第 i 种物品最多有 $n[i]$ 件可用, 每件容量是 $v[i]$, 价值是 $w[i]$ 。求解将哪些物品装入背包可使这些物品的费用总和不超过背包容量, 且价值总和最大。

这题目和完全背包问题很类似。基本的方程只需将完全背包问题的方程略微一改即可, 因为对于第 i 种物品有 $n[i] + 1$ 种策略: 取 0 件, 取 1 件……取 $n[i]$ 件。

令 $f[i][c]$ 表示前 i 种物品恰放入了容量为 c 的背包的最大权值, 则:

$$f[i][c] = \max f[i-1][c - k \times v[i]] + k \times w[i], f[i-1][c]$$

```

1  /* 多重背包：朴素动态规划 */
2  int multipleKnapsackDP(vector<int> &wgt, vector<int> &val, vector<int>
   ↪ &cnt, int cap) {
3      int n = wgt.size();
4      // 初始化 dp 表
5      vector<vector<int>> dp(n + 1, vector<int>(cap + 1, 0));
6
7      // 遍历物品
8      for (int i = 1; i <= n; i++) {
9          // 遍历背包容量
10         for (int c = 1; c <= cap; c++) {
11             // 朴素做法：枚举第 i-1 种物品选择的数量 k
12             // 限制条件：1. k 不超过该物品持有数量 cnt[i-1]
13             //                2. k 个该物品的总重不超过当前容量 c
14             for (int k = 0; k <= cnt[i - 1] && k * wgt[i - 1] <= c; k++)
15                 ↪ {
16                     dp[i][c] = max(dp[i][c], dp[i - 1][c - k * wgt[i - 1]] +
17                                     ↪ k * val[i - 1]);
18                 }
19         }
20     }
21     return dp[n][cap];
22 }

```

二进制优化的核心思想是：任何一个正整数 cnt 都可以拆分为若干个 $2^0, 2^1, 2^2, \dots$ 的组合。

通过这种拆分，我们可以把“一件件选物品”的过程，变成“一捆捆选物品”的过程。例如：如果你有 7 件相同的物品，你可以把它们打包成三捆，数量分别为 1, 2, 4。通过这三捆的组合，你可以凑出 0 到 7 之间的任何数量。这样，原本需要循环 7 次的多重背包就转化为了只需处理 3 个物品的 0-1 背包。

由 $O(V \times \sum n[i])$ 改进到 $O(V \times \sum \log(n[i]))$ 的过程：

```

1  /* 多重背包：二进制拆分优化 */
2  int binMultipleKnapsackDP(vector<int> &wgt, vector<int> &val,
   ↪ vector<int> &cnt, int cap) {
3      int n = wgt.size();
4      // 用于存储拆分后的新物品
5      vector<int> newWgt, newVal;
6

```

```

7      // 1. 拆分过程：将多重背包转化为 0-1 背包物品
8      for (int i = 0; i < n; i++) {
9          int num = cnt[i];
10         for (int k = 1; num > 0; k <= 1) { // k 依次为 1, 2, 4, 8...
11             if (num >= k) {
12                 newWgt.push_back(k * wgt[i]);
13                 newVal.push_back(k * val[i]);
14                 num -= k;
15             } else {
16                 // 处理剩余不足 2^n 的部分
17                 newWgt.push_back(num * wgt[i]);
18                 newVal.push_back(num * val[i]);
19                 num = 0;
20             }
21         }
22     }
23
24     // 2. 0-1 背包过程 (使用空间优化的滚动数组)
25     vector<int> dp(cap + 1, 0);
26     for (int i = 0; i < newWgt.size(); i++) {
27         // 0-1 背包必须倒序遍历
28         for (int c = cap; c >= newWgt[i]; c--) {
29             dp[c] = max(dp[c], dp[c - newWgt[i]] + newVal[i]);
30         }
31     }
32
33     return dp[cap];
34 }

```

单调队列优化是多重背包的最优解法，它将时间复杂度直接压低到 $O(n \cdot cap)$ 。
观察状态转移方程：

$$f[i][c] = \max f[i-1][c], f[i-1][c - k \cdot wgt] + k \cdot val$$

其中 $0 \leq k \leq cnt$ 。

如果我们令 $c = q \cdot wgt + r$ (其中 r 是余数, $0 \leq r < wgt$)，那么同一个物品在不同的容量下，只有余数 r 相同的状态之间才会相互转换。我们可以按余数 r 分组，利用单调队列维护一个长度为 $cnt + 1$ 的滑动窗口最大值。


```

1  int queueMultipleKnapsackDP(vector<int> &wgt, vector<int> &val,
    ↪ vector<int> &cnt, int cap) {
2      int n = wgt.size();
3      vector<int> dp(cap + 1, 0);
4      vector<int> pre(cap + 1, 0); // 记录上一轮物品的状态
5
6      for (int i = 0; i < n; i++) {
7          pre = dp; // 滚动数组备份
8          int w = wgt[i];
9          int v = val[i];
10         int num = cnt[i];
11
12         // 按余数 r 分组, r 范围是 [0, w-1]
13         for (int r = 0; r < w; r++) {
14             deque<int> dq; // 存储的是下标 (即在该余数分组下的第几个物品)
15             for (int q = 0; q * w + r <= cap; q++) {
16                 int curr_c = q * w + r;
17
18                 // 1. 维护队列单调性: 将当前状态值放入队列
19                 // 我们比较的是去掉“自带价值”后的净值: pre[curr_c] - q * v
20                 int val_net = pre[curr_c] - q * v;
21                 while (!dq.empty() && pre[dq.back() * w + r] - dq.back()
    ↪ * v <= val_net) {
22                     dq.pop_back();
23                 }
24                 dq.push_back(q);
25
26                 // 2. 过期出队: 如果最前面的下标超出了数量限制 num
27                 if (dq.front() < q - num) {
28                     dq.pop_front();
29                 }
30
31                 // 3. 更新状态: 当前最大值 + 这一路补回来的价值
32                 dp[curr_c] = max(dp[curr_c], pre[dq.front() * w + r] +
    ↪ (q - dq.front()) * v);
33             }
34         }
35     }

```

```

36     return dp[cap];
37 }

```

7.2 最长公共子序列 (Longest Common Subsequence)

设 $dp[i][j]$ 表示 S_a 的前 i 个字符与 S_b 前 j 个字符的最长公共子序列。转移方程：

$$dp[i][j] = \begin{cases} dp[i-1][j-1] + 1 & S_a[i] = S_b[j] \\ \max(dp[i-1][j], dp[i][j-1]) & S_a[i] \neq S_b[j] \end{cases}$$

```

1  int longestCommonSubsequence(string text1, string text2) {
2      int m = text1.length(), n = text2.length();
3      vector<vector<int>> dp(m + 1, vector<int>(n + 1));
4      for (int i = 1; i <= m; i++) {
5          char c1 = text1.at(i - 1);
6          for (int j = 1; j <= n; j++) {
7              char c2 = text2.at(j - 1);
8              if (c1 == c2) {
9                  dp[i][j] = dp[i - 1][j - 1] + 1;
10             } else {
11                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
12             }
13         }
14     }
15     return dp[m][n];
16 }

```

7.3 状态压缩 DP

遇到排列， $n \leq 20$ 大约这个数据范围的可以考虑。表示不重不漏地从点走完全程的最短路。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  array<array<int, 20>, 1 << 21> dp; //
   → dp[mask][j]: 经过 mask 集合中的点，当前停留在 j 点时的最小花费
5  array<array<int, 25>, 25> arr; // 邻接矩阵

```

```

6
7  int main() {
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     int n;
12     cin >> n;
13
14     // 初始化 DP 数组为无穷大
15     for (auto& i : dp) {
16         fill(i.begin(), i.end(), 0x3f3f3f3f);
17     }
18
19     // 读入地图数据
20     for (int i = 0; i < n; i++) {
21         for (int j = 0; j < n; j++) {
22             cin >> arr[i][j];
23         }
24     }
25
26     for (int i = 0; i < n; i++) {
27         dp[1 << i][i] = 0; // 只访问了节点 i, 且当前处于 i 点, 代价为 0
28     }
29
30     for (int i = 0; i < (1 << n); i++) {
31         for (int j = 0; j < n; j++) {
32             if ((i >> j) & 1) // 如果当前状态包含节点 j
33             {
34                 for (int k = 0; k < n; k++) {
35                     if (((i >> k) & 1) && k != j) // 从节点 k 转移到节点 j
36                     {
37                         dp[i][j] = min(dp[i][j], dp[i ^ (1 << j)][k] +
38                                         arr[k][j]);
39                     }
40                 }
41             }
42         }
43     }

```

```
43     int ans = 0x3f3f3f3f;
44     for (int i = 0; i < n; i++) {
45         ans = min(ans, dp[(1 << n) - 1][i]);
46     }
47
48     cout << ans << "\n";
49
50     return 0;
51 }
```

Chapter 8

常用函数重载

由于有些函数只会返回 `float` 所以在大数据的时候会造成一些精度丢失，所以如果确定操作后是整数，那么有下面的重载：

8.1 精确开平方

```
1  using i64 = long long;
2  i64 sqrt(i64 n) {
3      i64 s = std::sqrt(n);
4      while (s * s > n) {
5          s--;
6      }
7      while ((s + 1) * (s + 1) <= n) {
8          s++;
9      }
10     return s;
11 }
```

8.2 精准 log

```
1  int logi(int a, int b) {
2      int t = 0;
3      i64 v = 1;
4      while (v < b) {
5          v *= a;
6          t++;
7      }
8  }
```

```
8     return t;  
9 }
```

Chapter 9

字符串

9.1 前缀函数 (KMP)

给定一个长度为 n 的字符串 s ，其前缀函数被定义为一个长度为 n 的数组 π 。其中 $\pi[i]$ 的定义是：

1. 如果子串 $s[0 \dots i]$ 有一对相等的真前缀与真后缀： $s[0 \dots k-1]$ 和 $s[i-(k-1) \dots i]$ ，那么 $\pi[i]$ 就是这个相等的真前缀（或者真后缀，因为它们相等）的长度，也就是 $\pi[i] = k$ ；
2. 如果不止有一对相等的，那么 $\pi[i]$ 就是其中最长的那一对的长度；
3. 如果没有相等的，那么 $\pi[i] = 0$ 。

简单来说 $\pi[i]$ 就是，子串 $s[0 \dots i]$ 最长的相等的真前缀与真后缀的长度，下面的算法在 OI-WIKI 上证明复杂度为 $O(n)$

「你真要用到字符串匹配，直接 `str.find()` 省时省力，比你 Knuth - Morris - Pratt 还有 Boyer - Moore 什么算法花里胡哨的快得多甚至，所以一般不会用于字符串匹配。

——TimothySt」

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 using ll = long long;
4
5 // kmp 1-base
6 std::vector<int> kmp(std::string s) {
```

```
7     int n = s.size();
8     std::vector<int> f(n + 1);
9     for (int i = 1, j = 0; i < n; i++) {
10         while (j && s[i] != s[j]) {
11             j = f[j];
12         }
13         j += (s[i] == s[j]);
14         f[i + 1] = j;
15     }
16     return f;
17 }
18
19 vector<int> prefix_function(string s) {
20     int n = (int)s.length();
21     vector<int> pi(n);
22     for (int i = 1; i < n; i++) {
23         int j = pi[i - 1];
24         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
25         if (s[i] == s[j]) j++;
26         pi[i] = j;
27     }
28     return pi;
29 }
30
31 // 或者直接用 find_occurrences 封装度高
32
33 vector<int> find_occurrences(string text, string pattern) {
34     string cur = pattern + '#' + text;
35     int sz1 = text.size(), sz2 = pattern.size();
36     vector<int> v;
37     vector<int> lps = prefix_function(cur);
38     for (int i = sz2 + 1; i <= sz1 + sz2; i++) {
39         if (lps[i] == sz2) v.push_back(i - 2 * sz2);
40     }
41     return v;
42 }
43
44 // 朴素  $\theta$ -base  $O(n)$ 
```



```
45
46 vector<int> stringPi(string pattern) {
47     vector<int> pattern_pi(pattern.size());
48
49     for (int i = 1; i < pattern_pi.size(); i++) {
50         int len = pattern_pi[i - 1];
51         while (len != 0 && pattern[i] != pattern[len]) {
52             len = pattern_pi[len - 1];
53         }
54         if (pattern[i] == pattern[len]) {
55             pattern_pi[i] = len + 1;
56         }
57     }
58     return pattern_pi;
59 }
60
61 // 使用方法：查找 pattern 在 source 中的位置
62
63 int main() {
64     string source, pattern;
65     cin >> source >> pattern;
66     string fs = pattern + "#" + source;
67     int len = pattern.size();
68     auto fspi = kmp(fs);
69     for (int i = 1; i < fspi.size(); i++) {
70         if (fspi[i] == len) {
71             cout << i - len * 2 << "\n";
72         }
73         // cout << fspi[i] << " ";
74     }
75     auto pattpi = kmp(pattern);
76
77     for (int i = 1; i < pattpi.size(); i++) {
78         cout << pattpi[i] << " ";
79     }
80
81     return 0;
82 }
```

83

9.2 字典树 (Trie)

「个人感觉 Trie 没啥吊用。」

——TimothySt」

```
1 // https://www.luogu.com.cn/record/233659410
2 // By Timothy Starman 2025-08-29 10:38:42
3 #include <bits/stdc++.h>
4 using namespace std;
5
6 class TrieTree {
7 public:
8     int id;
9     char s = 0;
10    bool isrep = false;
11    vector<TrieTree*> vt;
12    vector<char> vc;
13    bool isWordEnd = false;
14 };
15 void TrieTreeInsert(TrieTree& root, string s) {
16     int sons = root.vc.size();
17     TrieTree* currentNode = &root;
18     int notFindIndex = 0;
19     bool allfind = true;
20     for (int i = 0; i < s.size(); i++) {
21         auto iter = find(currentNode->vc.begin(), currentNode->vc.end(),
22             ↪ s[i]);
23         if (iter == currentNode->vc.end()) {
24             notFindIndex = i;
25             allfind = false;
26             break;
27         }
28         int index = iter - currentNode->vc.begin();
29         currentNode = currentNode->vt[index];
30         if (i == s.size() - 1) {
```

```

30         currentNode->isWordEnd = true;
31     }
32 }
33 if (allfind == false) {
34     for (int j = notFindIndex; j < s.size(); j++) {
35         TrieTree* newTree = new TrieTree;
36         newTree->s = s[j];
37         currentNode->vt.push_back(newTree);
38         currentNode->vc.push_back(s[j]);
39         currentNode = newTree;
40         if (j == s.size() - 1) {
41             currentNode->isWordEnd = true;
42         }
43     }
44 }
45 }
46 string TrieTreeQuery(TrieTree& root, string s) {
47     int sons = root.vc.size();
48     TrieTree* currentNode = &root;
49     int notFindIndex = 0;
50     bool allfind = true;
51     for (int i = 0; i < s.size(); i++) {
52         auto iter = find(currentNode->vc.begin(), currentNode->vc.end(),
53             ↪ s[i]);
54         if (iter == currentNode->vc.end()) {
55             notFindIndex = i;
56             allfind = false;
57             return "WRONG";
58         }
59         int index = iter - currentNode->vc.begin();
60         currentNode = currentNode->vt[index];
61         if (i == s.size() - 1 && currentNode->isWordEnd == true &&
62             ↪ currentNode->isrep == false) {
63             currentNode->isrep = true;
64             return "OK";
65         } else if (i == s.size() - 1 && currentNode->isWordEnd == false)
66             ↪ {
67                 return "WRONG";
68             }
69     }
70 }

```

```

65         } else if (i == s.size() - 1 && currentNode->isrep == true) {
66             return "REPEAT";
67         }
68     }
69     return "WRONG";
70 }
71 int main() {
72     int t;
73     cin >> t;
74     TrieTree root;
75     for (int i = 0; i < t; i++) {
76         string s;
77         cin >> s;
78         TrieTreeInsert(root, s);
79     }
80     int q;
81     cin >> q;
82     for (int i = 0; i < q; i++) {
83         string s;
84         cin >> s;
85         cout << TrieTreeQuery(root, s) << "\n";
86     }
87
88     return 0;
89 }

```

9.3 AC 自动机 (Aho-Crasick)

```

1  #include <bits/stdc++.h>
2
3  class ACAutomaton {
4  private:
5      struct Node {
6          int next[26]; // 子节点指向
7          int fail;     // 失配指针
8          int count;    // 标记该节点结尾的模式串数量 (或 ID)
9
10         Node() {

```

```
11         memset(next, 0, sizeof(next));
12         fail = 0;
13         count = 0;
14     }
15 };
16
17 std::vector<Node> trie;
18
19 public:
20     ACAutomaton() {
21         trie.emplace_back(); // 初始化根节点 (index 0)
22     }
23
24     // 1. 插入模式串 (构建 Trie 树)
25     void insert(const std::string& s) {
26         int u = 0;
27         for (char ch : s) {
28             int v = ch - 'a';
29             if (!trie[u].next[v]) {
30                 trie[u].next[v] = trie.size();
31                 trie.emplace_back();
32             }
33             u = trie[u].next[v];
34         }
35         trie[u].count++; // 记录以该位置结尾的模式串个数
36     }
37
38     // 2. 构建 Fail 指针 (广度优先搜索 BFS)
39     void build() {
40         std::queue<int> q;
41         // 将根节点的子节点加入队列, 它们的 fail 指向根节点
42         for (int i = 0; i < 26; ++i) {
43             if (trie[0].next[i]) {
44                 q.push(trie[0].next[i]);
45             }
46         }
47
48         while (!q.empty()) {
```

```

49         int u = q.front();
50         q.pop();
51
52         for (int i = 0; i < 26; ++i) {
53             int v = trie[u].next[i];
54             if (v) {
55                 // 如果存在子节点 v, 则 v 的 fail 指向 u 的 fail 指向的对
56                 //   应子节点
57                 trie[v].fail = trie[trie[u].fail].next[i];
58                 q.push(v);
59             } else {
60                 // 路径压缩: 如果不存在子节点, 直接指向失配后的对应转移
61                 // 这一步将 Trie 树转变成了 Trie 图, 极大优化了查询速度
62                 trie[u].next[i] = trie[trie[u].fail].next[i];
63             }
64         }
65     }
66
67     // 3. 匹配文本并返回模式串出现的总次数
68     int query(const std::string& t) {
69         int u = 0, res = 0;
70         for (char ch : t) {
71             u = trie[u].next[ch - 'a']; // 沿 Trie 图转移
72             for (int j = u; j && trie[j].count != -1; j = trie[j].fail)
73                 {
74                     res += trie[j].count;
75                     trie[j].count = -1; // 标记已访问, 防止重复计算 (视题目需求而
76                                         //   定)
77                 }
78         }
79         return res;
80     }
81
82     // 重置 (用于多组测试数据)
83     void clear() {
84         trie.clear();
85         trie.emplace_back();

```

```
84     }
85 };
86
87 int main(int argc, char const* argv[]) {
88     int t;
89     std::cin >> t;
90
91     ACAutomaton ac;
92     for (int i = 0; i < t; i++) {
93         std::string tmp;
94         std::cin >> tmp;
95         ac.insert(tmp);
96     }
97     ac.build();
98     std::string q;
99     std::cin >> q;
100    std::cout << ac.query(q);
101
102    return 0;
103 }
```

Chapter 10

实用工具

10.1 .clang-format

个人使用压行规范。

```
1 BasedOnStyle: Google
2
3 IndentWidth: 4
4 TabWidth: 4
5 UseTab: Never
6
7 ContinuationIndentWidth: 4
8 AccessModifierOffset: -4
9
10 ColumnLimit: 100
11 # BreakBeforeBraces: Allman
```

10.2 关闭流同步

```
1 #include <bits/stdc++.h>
2 int main() {
3     std::ios::sync_with_stdio(false);
4     std::cin.tie(nullptr);
5     return 0;
6 }
7
```


10.3 代码计时器

配合数据生成器生成极端数据效果更佳，可以计算程序跑的时间。

```

1  struct Timer {
2      std::chrono::steady_clock::time_point start;
3      Timer() : start(std::chrono::steady_clock::now()) {}
4      ~Timer() {
5          auto finish = std::chrono::steady_clock::now();
6          auto runtime =
7              std::chrono::duration_cast<std::chrono::microseconds>(finish
8                  ↵ - start).count();
9              std::cerr << runtime / 1e6 << "s" << std::endl;
10     };
11 int main() {
12     int a;
13     std::cin >> a;
14     // 读入在前面完成或者使用重定向
15     Timer timer; // 从这里开始计时
16     std::set<int> S;
17     for (int i = 0; i < 1e6; ++i) {
18         S.insert(i);
19     }
20     return 0;
21 }
```

10.4 数据生成器

可以借此生成极端数据丢给计时器看运行时间。

```

1  template <typename T>
2  std::vector<T> genRandoms(T left, T right, int counts) {
3      std::random_device rd;
4      std::mt19937 generator(rd());
5      std::uniform_int_distribution<T> distribution(left, right);
6      std::vector<T> v;
7      for (int i = 0; i < counts; i++) {
8          v.push_back(distribution(generator));
9      }
```

```
9     }
10    return v;
11 }
12
13 int main() {
14     auto v = genRandoms(1ll, 1145141919810ll, 10);
15     for (auto i : v) {
16         std::cout << i << " ";
17     }
18     return 0;
19 }
```

10.5 Pair 哈希

```
1 using namespace std;
2
3 template <typename T1, typename T2>
4 struct PairHash {
5     size_t operator()(const pair<T1, T2>& p) const {
6         auto hash1 = hash<T1>{}(p.first);
7         auto hash2 = hash<T2>{}(p.second);
8
9         size_t seed = 0;
10        seed ^= hash1 + 0x9e3779b9 + (seed << 6) + (seed >> 2);
11        seed ^= hash2 + 0x9e3779b9 + (seed << 6) + (seed >> 2);
12
13        return seed;
14    }
15 };
16 struct UnorderedPairHash {
17     template <typename T>
18     std::size_t operator()(const std::pair<T, T>& p) const {
19         // 使用对称的哈希计算方式
20         auto h1 = std::hash<T>{}(std::min(p.first, p.second));
21         auto h2 = std::hash<T>{}(std::max(p.first, p.second));
22
23         // 合并两个哈希值，使用可交换的操作（如异或）
24         // 注意：直接异或可能导致碰撞，我们可以用更好的组合方式
```

```

25         return h1 ^ (h2 + 0x9e3779b9 + (h1 << 6) + (h1 >> 2));
26     }
27 };
28
29 struct UnorderedPairEqual {
30     template <typename T>
31     bool operator()(const std::pair<T, T>& lhs, const std::pair<T, T>&
32         ↪ rhs) const {
33         // 不考虑顺序的比较
34         return (lhs.first == rhs.first && lhs.second == rhs.second) ||
35             (lhs.first == rhs.second && lhs.second == rhs.first);
36     }
37 };
38
39 // 使用示例
40 std::unordered_set<std::pair<int, int>, UnorderedPairHash,
41     ↪ UnorderedPairEqual> mySet;
42 unordered_set<pair<int, int>, PairHash<int, int>> mySet;

```

10.6 Tuple 哈希

```

1  template <typename T>
2  void hash_combine(std::size_t& seed, const T& val) {
3      seed ^= std::hash<T>{}(val) + 0x9e3779b9 + (seed << 6) + (seed >>
4          ↪ 2);
5  }
6
7  template <typename... Types>
8  std::size_t hash_tuple(const std::tuple<Types...>& tuple) {
9      std::size_t seed = 0;
10     std::apply([&seed](const auto&... args) { (hash_combine(seed, args),
11         ↪ ...); }, tuple);
12     return seed;
13 }
14
15 namespace std {
16     template <typename... Types>
17     struct hash<std::tuple<Types...>> {

```

```

16     std::size_t operator()(const std::tuple<Types...& tuple) const {
17         ↪ return hash_tuple(tuple); }
18 };
19 // namespace std
20
21 int main() {
22     std::unordered_set<std::tuple<int, std::string, double>> mySet;
23
24     mySet.emplace(1, "hello", 3.14);
25     mySet.emplace(2, "world", 2.71);
26     mySet.emplace(1, "hello", 3.14); // 重复, 不会被插入
27
28     std::cout << "Set size: " << mySet.size() << std::endl; // 输出 2
29
30     return 0;
31 }

```

10.7 unordered 容器模数更改

```

1  #include <chrono>
2  #include <unordered_set>
3
4  struct custom_hash {
5      static uint64_t splitmix64(uint64_t x) {
6          // http://xorshift.di.unimi.it/splitmix64.c
7          x += 0x9e3779b97f4a7c15;
8          x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
9          x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
10         return x ^ (x >> 31);
11     }
12
13     size_t operator()(uint64_t x) const {
14         static const uint64_t FIXED_RANDOM =
15             std::chrono::steady_clock::now().time_since_epoch().count();
16         return splitmix64(x + FIXED_RANDOM);
17     }
18 };
19

```

```

20 // 使用方式:
21 std::unordered_set<long long, custom_hash> safe_set;
22 std::unordered_map<long long, int, custom_hash> safe_map;
23
24 #include <ext/pb_ds/assoc_container.hpp>
25 #include <ext/pb_ds/tree_policy.hpp>
26 using namespace __gnu_pbds;
27
28 typedef gp_hash_table<long long, int, custom_hash> safe_gp_hash_table;

```

10.8 字符串流

将字符串当作 stream 使用

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     string s = "12 34 56";
6     int a, b, c;
7     stringstream ss;
8     ss << s;           // 流式输出到 ss
9     ss >> a >> b >> c; // 流式输入给 a, b, c
10    return 0;
11 }

```

10.9 Lambda 自调用 (递归)

```

1 std::function<int(int const)> lfib = [&] fib(int const n) {
2     return n <= 2 ? 1 : lfib(n - 1) + lfib(n - 2);
3 };
4 auto f10 = lfib(10);

```

Chapter 11

Policy-Based Data Structures

11.1 GCC 扩展

pb_ds 库全称 Policy-Based Data Structures.

pb_ds 库封装了很多数据结构，比如哈希（Hash）表，平衡二叉树，字典树（Trie 树），堆（优先队列）等。

就像 vector、set、map 一样，其组件均符合 STL 的相关接口规范。部分（如优先队列）包含 STL 内对应组件的所有功能，但比 STL 功能更多。

pb_ds 只在使用 libstdc++ 为标准库的编译器下可以用。

可以使用 begin() 和 end() 来获取 iterator 从而遍历

可以 increase_key, decrease_key 以及删除单个元素

11.2 堆

保证 __gnu_pbds 与 std 冲突。

```
1 #include <ext/pb_ds/priority_queue.hpp>
2 using namespace __gnu_pbds;
3 __gnu_pbds::priority_queue<T, Compare, Tag, Allocator>
```

成员函数

```
1 #include <algorithm>
2 #include <cstdio>
3 #include <ext/pb_ds/priority_queue.hpp>
4 #include <iostream>
5 using namespace __gnu_pbds;
```

```

6  using pair_heap = __gnu_pbds::priority_queue<int>;
7  pair_heap q1;    // 大根堆, 配对堆
8  pair_heap q2;
9  pair_heap::point_iterator id;  // 一个迭代器
10
11  int main() {
12      id = q1.push(1);
13      // 堆中元素 : [1];
14      for (int i = 2; i <= 5; i++) q1.push(i);
15      // 堆中元素 : [1, 2, 3, 4, 5];
16      std::cout << q1.top() << std::endl;
17      // 输出结果 : 5;
18      q1.pop();
19      // 堆中元素 : [1, 2, 3, 4];
20      id = q1.push(10);
21      // 堆中元素 : [1, 2, 3, 4, 10];
22      q1.modify(id, 1);
23      // 堆中元素 : [1, 1, 2, 3, 4];
24      std::cout << q1.top() << std::endl;
25      // 输出结果 : 4;
26      q1.pop();
27      // 堆中元素 : [1, 1, 2, 3];
28      id = q1.push(7);
29      // 堆中元素 : [1, 1, 2, 3, 7];
30      q1.erase(id);
31      // 堆中元素 : [1, 1, 2, 3];
32      q2.push(1), q2.push(3), q2.push(5);
33      // q1 中元素 : [1, 1, 2, 3], q2 中元素 : [1, 3, 5];
34      q2.join(q1);
35      // q1 中无元素, q2 中元素 : [1, 1, 1, 2, 3, 3, 5];
36  }

```

迭代器的失效保证

```

1  #include <iostream>
2  using namespace std;
3  #include <ext/pb_ds/assoc_container.hpp>
4  #include <ext/pb_ds/priority_queue.hpp>
5  using namespace __gnu_pbds;

```

```

6  #include <cxxabi.h>
7
8  template <typename T>
9  void print_invalidation_guarantee() {
10     using gute =
        ↳ __gnu_pbds::container_traits<T>::invalidation_guarantee;
11     cout << abi::__cxa_demangle(typeid(gute).name(), 0, 0, 0) << endl;
12 }
13
14 int main() {
15     using pairing = __gnu_pbds::priority_queue<int, greater<int>,
        ↳ pairing_heap_tag>;
16     using binary = __gnu_pbds::priority_queue<int, greater<int>,
        ↳ binary_heap_tag>;
17     using binomial = __gnu_pbds::priority_queue<int, greater<int>,
        ↳ binomial_heap_tag>;
18     using rc_binomial = __gnu_pbds::priority_queue<int, greater<int>,
        ↳ rc_binomial_heap_tag>;
19     using thin = __gnu_pbds::priority_queue<int, greater<int>,
        ↳ thin_heap_tag>;
20     print_invalidation_guarantee<pairing>();
21     print_invalidation_guarantee<binary>();
22     print_invalidation_guarantee<binomial>();
23     print_invalidation_guarantee<rc_binomial>();
24     print_invalidation_guarantee<thin>();
25     return 0;
26 }

```

11.3 平衡树

构造类型

```

1  #include <ext/pb_ds/assoc_container.hpp> // 因为 tree 定义在这里 所以需要包
    ↳ 含这个头文件
2  #include <ext/pb_ds/tree_policy.hpp>
3  using namespace __gnu_pbds;
4  __gnu_pbds::tree<Key, Mapped, Cmp_Fn = std::less<Key>, Tag =
    ↳ rb_tree_tag,

```



```

5         Node_Update = null_tree_node_update, Allocator =
        ↪ std::allocator<char>>

```

成员函数

```

1  // Common Header Simple over C++11
2  #include <iostream>
3  using namespace std;
4  using ll = long long;
5  using ull = unsigned long long;
6  using ld = long double;
7  using pii = pair<int, int>;
8  #include <ext/pb_ds/assoc_container.hpp>
9  #include <ext/pb_ds/tree_policy.hpp>
10 __gnu_pbds::tree<pair<int, int>, __gnu_pbds::null_type, less<pair<int,
    ↪ int>>,
11         __gnu_pbds::rb_tree_tag,
    ↪ __gnu_pbds::tree_order_statistics_node_update>
12     trr;
13
14 int main() {
15     int cnt = 0;
16     trr.insert(make_pair(1, cnt++));
17     trr.insert(make_pair(5, cnt++));
18     trr.insert(make_pair(4, cnt++));
19     trr.insert(make_pair(3, cnt++));
20     trr.insert(make_pair(2, cnt++));
21     // 树上元素 {(1,0), (2,4), (3,3), (4,2), (5,1)}
22
23     auto it = trr.lower_bound(make_pair(2, 0));
24     trr.erase(it);
25     // 树上元素 {(1,0), (3,3), (4,2), (5,1)}
26
27     // 输出排名 0 1 2 3 中的排名 1 的元素的 first
28     auto it2 = trr.find_by_order(1);
29     cout << (*it2).first << endl; // 输出: 3
30
31     // 输出其排名
32     int pos = trr.order_of_key(*it2);

```

```
33     cout << pos << endl; // 输出: 1
34
35     // 按照 it2 分裂 trr
36     decltype(trr) newtr;
37     trr.split(*it2, newtr);
38     for (auto i = newtr.begin(); i != newtr.end(); ++i) {
39         cout << (*i).first << ' '; // 输出: 4 5
40     }
41     cout << endl;
42
43     // 将 newtr 树并入 trr 树, newtr 树被清空.
44     trr.join(newtr);
45     for (auto i = trr.begin(); i != trr.end(); ++i) {
46         cout << (*i).first << ' '; // 输出: 1 3 4 5
47     }
48     cout << endl;
49     cout << newtr.size() << endl; // 输出: 0
50
51     return 0;
52 }
```